ID: 2602050330

2026-2030-Batch2-I1

Manipal University Jaipur

S.No: 1	Exp. Name: ***Time Conversion***	**Date: 2026-09-30**

Aim:

Write a program to convert the time in seconds to hours, minutes, and seconds. (1 hr = 3600 sec).

Input Format:

- The program should display the prompt:

```
Enter the time in Seconds:
```

- The user should enter a single integer representing the time in seconds on the same line.

Output Format:

- The program should display the time in hours, minutes, and seconds in the following format:

```
Time = <hours> Hours <minutes> Minutes <seconds> Seconds
```

Source Code:

SmartwatchTime.c

```
#include <stdio.h>

int main(void)
{
        int time, m, h, s;
        printf("Enter the time in Seconds: ");
        scanf("%d", &time);

        h = time/3600;
        m = (time%3600)/60;
        s = time%60;

        printf("Time = %d Hours %d Minutes %d Seconds", h, m, s);

}
```

Execution Results - All test cases have succeeded!

Test Case - 1

User Output
Enter the time in Seconds:
3665
Time = 1 Hours 1 Minutes 5 Seconds

Test Case - 2
User Output
Enter the time in Seconds:
0
Time = 0 Hours 0 Minutes 0 Seconds

Test Case - 3
User Output
Enter the time in Seconds:
200
Time = 0 Hours 3 Minutes 20 Seconds

ID: 2602050330

ID: 2602050330

2026-2030-Batch2-I1

Manipal University Jaipur

S.No: 2	Exp. Name: ***Sum of Digits***	**Date: 2026-09-30**

Aim:

Write a program to find the sum of the digits of a four-digit number (ex. 1234 sum=10) without using a loop.

Input Format:

- The program should display the prompt:

```
Enter four digit number
```

- The user should enter a single four-digit integer representing the number on the same line.

Output Format:

- The program should display the sum of the four digits in the following format:

```
Sum of Digits = <sum>
```

Source Code:

sum_of_digits.c

```
#include <stdio.h>

int main(void)
{
        int n, sum=0;
        printf("Enter four digit number ");
        scanf("%d",&n);
        sum+=n%10;
        n=n/10;
        sum+=n%10;
        n=n/10;
        sum+=n%10;
        n=n/10;
        sum+=n%10;

        printf("Sum of Digits = %d", sum);
        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter four digit number
2456
Sum of Digits = 17

Test Case - 2
User Output
Enter four digit number
9999
Sum of Digits = 36

ID: 2602050330

2026-2030-Batch2-I1

Manipal University Jaipur

S.No: 3	Exp. Name: ***Temperature Conversion Between Celsius and Fahrenheit***	**Date: 2026-09-30**

Aim:

Write a program to convert the temperature given in Fahrenheit to Centigrade and Centigrade to Fahrenheit.

Hint:

- $C = \left(\frac{5.0}{9.0}(F - 32)\right)$.
- $F = \left(\frac{9.0}{5.0}\right)C + 32$

Input Format:

- The program should display the prompt:

```
Enter temperature in Fahrenheit:
```

- The user should enter a single floating-point value representing the temperature in Fahrenheit on the same line.

Output Format:

- The program should display the converted temperature in Celsius in the following format:

```
Temperature in Celsius = <celsius>
```

- The program should then display the converted temperature in Fahrenheit in the following format:

```
Temperature in Fahrenheit = <fahrenheit>
```

Source Code:

celsiusFahrenheit.c

```
#include <stdio.h>

int main()
{
        float temp,C,F;

        printf("Enter temperature in Fahrenheit: ");
        scanf("%f", &temp);
        C=((5.0/9.0)*(temp-32));
        F=(C*(9.0/5.0)) + 32.0;
        printf("Temperature in Celsius = %f\n",C);
        printf("Temperature in Fahrenheit = %f\n",F);
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter temperature in Fahrenheit:
77
Temperature in Celsius = 25.000000
Temperature in Fahrenheit = 77.000000

Test Case - 2
User Output
Enter temperature in Fahrenheit:
212.1
Temperature in Celsius = 100.055557
Temperature in Fahrenheit = 212.100006

S.No: 4	Exp. Name: ***Distance Conversion***	Date: 2026-09-30

Aim:

Write a program for converting distance in mm to cm, inch, feet (1 cm =10mm, 1 inch=2.54cm, 1 feet =12 inches).

Input Format:

- The program should display the prompt:

```
Enter distance in millimeters:
```

- The user should enter a single floating-point value representing the distance in millimeters on the same line.

Output Format:

- The program should display the equivalent distance in centimeters, inches and feet in the following format:

```
Centimeters = <centimeters>
Inches = <inches>
Feet = <feet>
```

Source Code:

distanceConvert.c

```
#include <stdio.h>

int main()
{
        float mm, cm, inch, feet;
        printf("Enter distance in millimeters:");
        scanf("%f", &mm);

        cm = mm/10;
        inch = cm/2.54;
        feet = inch/12;

        printf("Centimeters = %f\nInches = %f\nFeet = %f", cm, inch,
feet);
        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter distance in millimeters:
250
Centimeters = 25.000000
Inches = 9.842520
Feet = 0.820210

Test Case - 2
User Output
Enter distance in millimeters:
1000
Centimeters = 100.000000
Inches = 39.370079
Feet = 3.280840

S.No: 5	Exp. Name: ***Euclidean Distance Calculation***	**Date: 2026-09-30**

Aim:

Write a C program to calculate the Euclidean distance between two points in a 2D plane, given their coordinates (x_1, y_1) and (x_2, y_2).

Formula:

Distance= $\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$

Input Format:

- The program should display the prompt "Enter x1 and y1: ".
- The user should enter two floating-point values representing the x_1 and y_1 coordinates, separated by a space, on the same line.
- The program should then display the prompt "Enter x2 and y2: ".
- The user should enter two floating-point values representing the x_2 and y_2 coordinates, separated by a space, on the same line.

Output Format:

- The program should display the distance between the two points in the following format:

```
Distance = <distance>
```

Source Code:

distance.c

```
#include <stdio.h>
#include <math.h>
int main()
{
        float x1, y1, x2, y2, d;
        printf("Enter x1 and y1: ");
        scanf("%f%f", &x1, &y1);
        printf("Enter x2 and y2: ");
        scanf("%f%f", &x2, &y2);
        d=sqrt((x2-x1) * (x2-x1) + (y2-y1) * (y2-y1));
        printf("Distance = %f", d);
        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter x1 and y1:
0 0
Enter x2 and y2:
5 0
Distance = 5.000000

Test Case - 2
User Output
Enter x1 and y1:
0 0
Enter x2 and y2:
0 4
Distance = 4.000000

Test Case - 3
User Output
Enter x1 and y1:
0 0
Enter x2 and y2:
3 4
Distance = 5.000000

Test Case - 4
User Output
Enter x1 and y1:
0 0
Enter x2 and y2:
0 0
Distance = 0.000000

Test Case - 5

-2 3
Enter x2 and y2:
4 -1
Distance = 7.211102

ID: 2602050330

2026-2030-Batch2-I1

Manipal University Jaipur

S.No: 6	Exp. Name: ***Area of Circle***	**Date: 2026-09-30**

Aim:

Write a program to evaluate the area of the circle using the formula

$$Area = pi \times R^2$$

where $pi = 3.142$

Input Format:

- The program should display the prompt:

```
Enter the radius of the circle:
```

- The user should enter a single floating-point value representing the radius of the circle R on the same line.

Output Format:

- The program should display the area of the circle in the following format:

```
Area of Circle = <area>
```

Note: The area should be displayed as a floating-point value using the default `%f` format, with six digits after the decimal point.

Source Code:

Area.c

```
#include <stdio.h>

int main()
{
        float area, pi=3.142, r;
        printf("Enter the radius of the circle: ");
        scanf("%f", &r);
        area=pi*r*r;
        printf("Area of Circle = %f\n", area);
        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1

User Output
Enter the radius of the circle:
7.5
Area of Circle = 176.737503

Test Case - 2
User Output
Enter the radius of the circle:
10.0
Area of Circle = 314.200012

S.No: 7	Exp. Name: ***Swap Two Numbers using a Third Variable***	**Date: 2026-09-11**

Aim:

Write a program to interchange the values of two variables using a third variable.

Input Format:

- The program should display the prompt:

```
Enter the two numbers:
```

- The user should enter two integers separated by a space on the same line.

Output Format:

- The program should display the message:

```
After Swapping:
```

- The program should then display the value of a in the following format:

```
a= <value of a>
```

- The program should display the value of b in the following format:

```
b= <value of b>
```

Source Code:

swapwithoutvar.c

```
#include <stdio.h>

int main()
{
        int a, b, c;
        printf("Enter the two numbers: ");
        scanf("%d%d", &a, &b);
        c = a;
        a = b;
        b = c;
        printf("After Swapping:\n");
        printf("a= %d\n", a);
        printf("b= %d\n", b);
        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter the two numbers:
12 18
After Swapping:
a= 18
b= 12

Test Case - 2
User Output
Enter the two numbers:
90 56
After Swapping:
a= 56
b= 90

ID: 2602050330

2026-2030-Batch2-I1

Manipal University Jaipur

S.No: 8	Exp. Name: ***Swap Two Numbers***	**Date: 2026-09-11**

Aim:

Write a program to interchange the values of two variables without using a third variable

Input Format:

- The program should display the prompt:

```
Enter the two numbers:
```

- The user should enter two integers separated by a space on the same line.

Output Format:

- The program should display the message:

```
After Swapping:
```

- The program should then display the value of a in the following format:

```
a= <value of a>
```

- The program should display the value of b in the following format:

```
b= <value of b>
```

Source Code:

swapwithoutvar.c

```
#include <stdio.h>

int main()
{
        int a,b;
        printf("Enter the two numbers: ");
        scanf("%d%d",&a,&b);
        a=a+b;
        b=a-b;
        a=a-b;
        printf("After Swapping:\n");
        printf("a= %d\nb= %d\n", a, b);
        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter the two numbers:
12 18
After Swapping:
a= 18
b= 12

Test Case - 2
User Output
Enter the two numbers:
90 56
After Swapping:
a= 56
b= 90

S.No: 9	Exp. Name: ***Checking Whether a Number is Even or Odd***	**Date: 2026-09-30**

Aim:

Write a program to check whether the given number is odd or even.

Input Format:

• The program should display the prompt:

```
Enter the number:
```

• The user should enter a single integer num on the same line as the prompt.

Output Format:

• If the input number is even, the program should display:

```
The input number <num> is Even
```

• Otherwise, the program should display:

```
The input number <num> is Odd
```

• where num is the integer entered by the user.·

Source Code:

EvenOdd.c

```
#include <stdio.h>

int main()
{
        int num;
        printf("Enter the number: ");
        scanf("%d", &num);
        if (num%2==0)
                printf("The input number %d is Even", num);
        else
                printf("The input number %d is Odd", num);
        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter the number:

The input number 0 is Even

Test Case - 2
User Output
Enter the number:
24
The input number 24 is Even

Test Case - 3
User Output
Enter the number:
-15
The input number -15 is Odd

Test Case - 4
User Output
Enter the number:
7
The input number 7 is Odd

S.No: 10	Exp. Name: ***Checking Whether a Year is a Leap Year***	**Date: 2026-09-30**

Aim:

Write a program to check whether a given year is a leap year or not.

Input Format:

- The program should display the prompt:

```
Enter the Year:
```

- The user should enter a single integer $year$ representing the year to be checked, on the same line as the prompt.

Output Format:

- If the input year is a leap year, the program should display:

```
The input year <year> is Leap year
```

- Otherwise, the program should display:

```
The input year <year> is not Leap year
```

- where $year$ is the integer entered by the user.

Source Code:

leap.c

```
#include <stdio.h>

int main()
{
        int year;
        printf("Enter the Year: ");
        scanf("%d", &year);
        if((year%4==0 && year%100!=0) || (year%400==0))
                printf("The input year %d is Leap year", year);
        else
                printf("The input year %d is not Leap year", year);
        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter the Year:

2000
The input year 2000 is Leap year

Test Case - 2
User Output
Enter the Year:
1900
The input year 1900 is not Leap year

Test Case - 3
User Output
Enter the Year:
2023
The input year 2023 is not Leap year

S.No: 11	Exp. Name: ***Finding the Roots of a Quadratic Equation***	**Date: 2026-09-30**

ID: 2602050330

2026-2030-Batch2-I1

Manipal University Jaipur

Aim:

Write a program to find the roots of a quadratic equation Hint:

$$\text{root} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Input Format:

- The program should display the prompt:

```
Enter the values of a, b and c:
```

- Enter three floating-point values a, b, and c, separated by spaces, on the same line as the prompt. These values represent the coefficients of the quadratic equation: $ax^2 + bx + c = 0$

Output Format:

- If the discriminant is greater than 0, the program should display the two distinct real roots in the following format:

```
Root 1 = root1
Root 2 = root2
```

- where $root1$ and $root2$ are displayed with two decimal places.
- If the discriminant is equal to 0, the program should display:

```
Both roots are equal
Root 1 = Root 2 = <root>
```

- where $root$ is displayed with two decimal places.
- If the discriminant is less than 0, the program should display:

```
Roots are imaginary
```

Source Code:

QuadraticEquation.c

```c
#include <stdio.h>
#include <math.h>

int main()
{
        float a,b,c,d, root, root1, root2;
        printf("Enter the values of a, b and c: ");
        scanf("%f%f%f", &a, &b, &c);

        d=b*b-4*a*c;
        if (d==0)
        {
                root=-b/(2*a);
                printf("Both roots are equal\n");
                printf("Root 1 = Root 2 = %.2f", root);
        }
        else if (d>0)
        {
                root1=(-b+sqrt(d))/(2*a);
                root2=(-b-sqrt(d))/(2*a);
                printf("Root 1 = %.2f\n", root1);
                printf("Root 2 = %.2f\n", root2);
        }
        else
                printf("Roots are imaginary");
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter the values of a, b and c:
1 -5 6
Root 1 = 3.00
Root 2 = 2.00

Test Case - 2
User Output

Enter the values of a, b and c:
1 -4 4
Both roots are equal
Root 1 = Root 2 = 2.00

Test Case - 3
User Output
Enter the values of a, b and c:
1 2 5
Roots are imaginary

ID: 2602050330
2026-2030-Batch2-I1
Manipal University Jaipur

S.No: 12	Exp. Name: ***Total Number of Days in the First N Months of a Year***	**Date: 2026-09-30**

ID: 2602050330

2026-2030-Batch2-I1

Manipal University Jaipur

Aim:

Write a program to find the total no. of days for a given number of months counting from January month. Example: m=3, days=31+ (28 0r 29) + 31.

Input Format:

- The program should display the prompt:

```
Enter the number of months:
```

- The user should enter a single integer $month$, representing the number of months to be considered, on the same line as the prompt.
- The program should then display the prompt:

```
Enter the year:
```

- The user should enter a single integer $year$ representing the year to be considered, on the same line as the prompt.

Output Format:

- The program should display the total number of days in the specified number of months in the following format:

```
Total number of days = <days>
```

- where $days$ represents the total number of days calculated for the first $month$ months of the given $year$.

Source Code:

DaysCount.c

```
#include <stdio.h>

int main()
{
        int year, month, days=0;

        printf("Enter the number of months: ");
        scanf("%d", &month);
        printf("Enter the year: ");
        scanf("%d", &year);
        int feb=28;
        if ((year%4==0 && year%100!=0) || (year%400==0))
                feb=29;
        if (month>=1) days+=31;
        if (month>=2) days+=feb;
        if (month>=3) days+=31;
        if (month>=4) days+=30;
        if (month>=5) days+=31;
        if (month>=6) days+=30;
        if (month>=7) days+=31;
        if (month>=8) days+=31;
        if (month>=9) days+=30;
        if (month>=10) days+=31;
        if (month>=11) days+=30;
        if (month>=12) days+=31;

        printf("Total number of days = %d", days);
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter the number of months:
1
Enter the year:
2024
Total number of days = 31

Test Case - 2
User Output
Enter the number of months:
2
Enter the year:
2024
Total number of days = 60

Test Case - 3
User Output
Enter the number of months:
2
Enter the year:
2023
Total number of days = 59

S.No: 13	Exp. Name: ***Two Numbers Are Multiples of Each Other***	**Date: 2026-09-30**

Aim:

Write a program to take two numbers as input and find whether one number is a multiple of the other or not.

Input Format:

- The program should display the prompt:

```
Enter two numbers:
```

- The user should enter two integers $num1$ and $num2$, separated by a space, on the next line after the prompt.

Output Format:

- If $num2$ is 0, the program should display:

```
Division by zero is not possible
```

- If $num1$ is a multiple of $num2$, the program should display:

```
<num1> is a multiple of <num2>
```

- Otherwise, if $num2$ is a multiple of $num1$, the program should display:

```
<num2> is a multiple of <num1>
```

- If neither number is a multiple of the other, the program should display:

```
Neither number is a multiple of the other
```

Source Code:

Multiples.c

```
#include <stdio.h>

int main()
{
        int num1, num2;
        printf("Enter two numbers:\n");
        scanf("%d%d", &num1, &num2);
        if (num2 == 0)
                printf("Division by zero is not possible");
        else if (num1 % num2==0)
                printf("%d is a multiple of %d", num1, num2);
        else if (num2 % num1==0)
                printf("%d is a multiple of %d", num2, num1);
        else
                printf("Neither number is a multiple of the other");
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter two numbers:
10 0
Division by zero is not possible

Test Case - 2
User Output
Enter two numbers:
20 5
20 is a multiple of 5

Test Case - 3
User Output
Enter two numbers:
7 3
Neither number is a multiple of the other

ID: 2602050330
2026-2030-Batch2-I1
Manipal University Jaipur

S.No: 14	Exp. Name: ***Determining Quiz Grade Based on Score***	**Date: 2026-09-30**

ID: 2602050330

2026-2030-Batch2-I1

Manipal University Jaipur

Aim:

Write a program that quiz. The letter grades are assigned as:

- 9-10 -> "A"
- 7-8 -> "B"
- 5-6 -> "C"
- 3-4 -> "D"
- <3 -> "F"

Input Format:

- The program should display the prompt:

```
Enter quiz score (0-10):
```

- The user should enter a single integer score representing the quiz score, on the same line as the prompt.

Output Format:

- If the entered score is less than 0 or greater than 10, the program should display:

```
Invalid score
```

- If the score is between 0 and 10, the program should display the corresponding grade in the following format:

```
Grade = <grade>
```

where $grade$ is:

- A for scores from 9 to 10
- B for scores from 7 to 8
- C for scores from 5 to 6
- D for scores from 3 to 4
- F for scores from 0 to 2

Source Code:

Qiuz.c

```
#include <stdio.h>

int main()
{
        int grade;
        printf("Enter quiz score (0-10): ");
        scanf("%d", &grade);

        if (grade < 0 || grade > 10)
                printf("Invalid score");
        else if (grade>=9)
                printf("Grade = A");
        else if (grade>=7)
                printf("Grade = B");
        else if (grade >= 5)
                printf("Grade = C");
        else if (grade >= 3)
                printf("Grade = D");
        else
                printf("Grade = F");
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter quiz score (0-10):
0
Grade = F

Test Case - 2
User Output
Enter quiz score (0-10):
10
Grade = A

User Output
Enter quiz score (0-10):
9
Grade = A

Test Case - 4
User Output
Enter quiz score (0-10):
6
Grade = C

Test Case - 5
User Output
Enter quiz score (0-10):
-2
Invalid score

Test Case - 6
User Output
Enter quiz score (0-10):
11
Invalid score

S.No: 15	Exp. Name: ***Validating a Triangle and Calculating Its Area***	**Date: 2026-09-30**

ID: 2602050330

2026-2030-Batch2-I1

Manipal University Jaipur

Aim:

Write a program that takes three sides of a triangle as input and checks whether they form a valid triangle using the following conditions: $a + b > c$, $b + c > a$ and $a + c > b$
If the sides form a valid triangle, calculate the semi-perimeter using:

$$s = (a + b + c)/2$$

Then calculate the area of the triangle using Heron's formula:

$$Area = \sqrt{(s(s - a)(s - b)(s - c))}$$

Input Format:

- The program should display the prompt:

```
Enter three sides of the triangle:
```

- The user should enter three floating-point values a, b, and c, representing the three sides of the triangle, separated by spaces, on the same line as the prompt.

Output Format:

- If the given sides form a valid triangle, the program should display:

```
The triangle is valid
Area of triangle = area
```

- The area should be displayed with two decimal places.
- If the given sides do not form a valid triangle, the program should display:

```
The given sides do not form a valid triangle
```

Source Code:

Area.c

```
#include <stdio.h>
#include <math.h>

int main()
{
        float a, b, c, s, area;
        printf("Enter three sides of the triangle: ");
        scanf("%f%f%f", &a, &b, &c);

        if (a>0 && b>0 && c>0 && (a+b>c) && (b+c>a) && (a+c>b))
        {
                s=(a+b+c)/2.0;
                area=sqrt(s*(s-a)*(s-b)*(s-c));
                printf("The triangle is valid\n");
                printf("Area of triangle = %.2f", area);
        }
        else
                printf("The given sides do not form a valid triangle");
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter three sides of the triangle:
3 4 5
The triangle is valid
Area of triangle = 6.00

Test Case - 2
User Output
Enter three sides of the triangle:
1 2 3
The given sides do not form a valid triangle

Test Case - 3

User Output
Enter three sides of the triangle:
2.5 3.5 4
The triangle is valid
Area of triangle = 4.33

ID: 2602050330

2026-2030-Batch2-I1

Manipal University Jaipur

S.No: 16	Exp. Name: ***Checking Character***	Date: 2026-09-30

Aim:

Write a program to check whether the given character is a vowel, consonant, or digit.

Input Format:

- The program should display the prompt:

`Enter a character:`

- The user should enter a single character ch on the same line as the prompt.

Output Format:

- If the entered character is a digit, the program should display:

`<ch> is a digit`

- If the entered character is a vowel, the program should display:

`<ch> is a vowel`

- If the entered character is an alphabetic character but not a vowel, the program should display:

`<ch> is a consonant`

- Otherwise, the program should display:

`<ch> is a unknown character`

- where ch is the character entered by the user.

Source Code:

CharCheck.c

```
#include <stdio.h>

int main()
{
        char ch;
        printf("Enter a character: ");
        scanf(" %c", &ch);

        if (ch>='0' && ch<='9')
                printf("%c is a digit", ch);
        else if (ch=='a' || ch=='A' || ch=='e' || ch=='E' || ch=='i' ||
ch=='I' || ch=='o' || ch=='O' || ch=='u' || ch=='U')
                printf("%c is a vowel", ch);
        else if ((ch>='a' && ch<='z') || (ch>='A' && ch<='Z'))
                printf("%c is a consonant", ch);
        else
                printf("%c is a unknown character", ch);
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter a character:
7
7 is a digit

Test Case - 2
User Output
Enter a character:
A
A is a vowel

Test Case - 3
User Output
Enter a character:
@
@ is a unknown character

Test Case - 4
User Output
Enter a character:
m
m is a consonant

ID: 2602050330

2026-2030-Batch2-I1

Manipal University Jaipur

S.No: 17	Exp. Name: ***Area Calculation Using Switch-Case in C***	**Date: 2026-09-30**

Aim:

Write a C program to calculate the area of a circle, rectangle, or triangle based on the user's choice.

Input Format:

1. The first input is an integer representing the user's choice:
   - 1 – Area of Circle
   - 2 – Area of Rectangle
   - 3 – Area of Triangle
2. If the choice is 1, enter the radius(float).
3. If the choice is 2, enter the space separated floating point values that represents length and breadth.
4. If the choice is 3, enter the space separated floating point values that represents base and height.

Output Format:

1. For choice 1, display the area of the circle in the format:

```
Area of Circle = <area rounded to 2 decimal places>
```

2. For choice 2, display the area of the rectangle in the format:

```
Area of Rectangle = <area rounded to 2 decimal places>
```

3. For choice 3, display the area of the triangle in the format:

```
Area of Triangle = <area rounded to 2 decimal places>
```

4. If the user enters any choice other than 1, 2, or 3, display:

```
Invalid choice
```

Source Code:

AreaCalculation.c

```
#include <stdio.h>

int main()
{
        int choice;
        float r, l, w, b, h, area;
        printf("1. Area of Circle\n2. Area of Rectangle\n3. Area of
Triangle\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch(choice)
        {
                case 1:
                        printf("Enter radius: ");
                        scanf("%f", &r);
                        area=3.14*r*r;
                        printf("Area of Circle = %.2f", area);
                        break;
                case 2:
                        printf("Enter length and breadth: ");
                        scanf("%f%f", &l, &w);
                        area=l*w;
                        printf("Area of Rectangle = %.2f", area);
                        break;
                case 3:
                        printf("Enter base and height: ");
                        scanf("%f%f", &b, &h);
                        area=0.5*b*h;
                        printf("Area of Triangle = %.2f", area);
                        break;
                default:
                        printf("Invalid choice");
                        break;
                }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output

1. Area of Circle
2. Area of Rectangle
3. Area of Triangle
Enter your choice:
2
Enter length and breadth:
10 5
Area of Rectangle = 50.00

Test Case - 2
User Output
1. Area of Circle
2. Area of Rectangle
3. Area of Triangle
Enter your choice:
3
Enter base and height:
8 6
Area of Triangle = 24.00

Test Case - 3
User Output
1. Area of Circle
2. Area of Rectangle
3. Area of Triangle
Enter your choice:
5
Invalid choice

Test Case - 4
User Output
1. Area of Circle
2. Area of Rectangle
3. Area of Triangle
Enter your choice:
1

```
Area of Circle = 19.62
```

S.No: 18	Exp. Name: ***Switch-Case Without Break Statements in C***	**Date: 2026-09-30**

ID: 2602050330

2026-2030-Batch2-I1

Manipal University Jaipur

Aim:

Write a C program to remove all the `break` statements from the previous question - "Write a C program to calculate the area of a circle, rectangle, or triangle based on the user's choice" using a switch-case program and execute it with different inputs. Observe and understand the difference in program execution due to fall-through behavior in a switch-case statement.

Input Format:

1. The first input is an integer representing the user's choice:
    - 1 – Area of Circle
    - 2 – Area of Rectangle
    - 3 – Area of Triangle
2. If the choice is 1, enter the radius(float).
3. If the choice is 2, enter the space separated floating point values that represents length and breadth.
4. If the choice is 3, enter the space separated floating point values that represents base and height.

Output Format:

1. For choice 1, display the area of the circle in the format:

```
Area of Circle = <area rounded to 2 decimal places>
```

2. For choice 2, display the area of the rectangle in the format:

```
Area of Rectangle = <area rounded to 2 decimal places>
```

3. For choice 3, display the area of the triangle in the format:

```
Area of Triangle = <area rounded to 2 decimal places>
```

4. Since the **break** statements are removed, execution continues into the subsequent cases until the **switch** statement ends.
5. For an invalid choice, display"End of program"

Source Code:

withoutbreak.c

```
#include <stdio.h>

int main()
{
        int choice;
        float r, l, w, b, h, area;
        printf("1. Circle\n2. Rectangle\n3. Triangle\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch(choice)
        {
                case 1:
                        printf("Enter radius: ");
                        scanf("%f", &r);
                        area=3.14*r*r;
                        printf("Area of Circle = %.2f\n", area);
                case 2:
                        printf("Enter length and breadth: ");
                        scanf("%f%f", &l, &w);
                        area=l*w;
                        printf("Area of Rectangle = %.2f\n", area);
                case 3:
                        printf("Enter base and height: ");
                        scanf("%f%f", &b, &h);
                        area=0.5*b*h;
                        printf("Area of Triangle = %.2f\n", area);
                default:
                        printf("End of program");
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
1. Circle
2. Rectangle
3. Triangle
Enter your choice:
1

ID: 2602050330
2026-2030-Batch2-I1
Manipal University Jaipur

Enter radius:
5
Area of Circle = 78.50
Enter length and breadth:
10 4
Area of Rectangle = 40.00
Enter base and height:
8 6
Area of Triangle = 24.00
End of program

Test Case - 2
User Output
1. Circle
2. Rectangle
3. Triangle
Enter your choice:
3
Enter base and height:
12.5 13.6
Area of Triangle = 85.00
End of program

Test Case - 3
User Output
1. Circle
2. Rectangle
3. Triangle
Enter your choice:
2
Enter length and breadth:
10.5 12.2
Area of Rectangle = 128.10
Enter base and height:
10.2 20.5
Area of Triangle = 104.55
End of program

ID: 2602050330

2026-2030-Batch2-I1

Manipal University Jaipur

S.No: 19	Exp. Name: ***Convert Day Number to Day Name Using Switch-Case in C***	**Date: 2026-09-30**

Aim:

Write a C program to input a number from 1 to 7 and display the corresponding day of the week using a switch-case statement.

The day numbers are mapped as follows:

- 1 → Sunday
- 2 → Monday
- 3 → Tuesday
- 4 → Wednesday
- 5 → Thursday
- 6 → Friday
- 7 → Saturday

For any number outside the range 1-7, display "Invalid day number".

Input Format:

- The program displays the prompt:

```
Enter day number (1-7):
```

- Enter an integer representing the day number from 1 to 7.

Output Format:

- If the input is between 1 and 7, print the corresponding day name.
- If the input is not between 1 and 7, print:

```
Invalid day number
```

Source Code:

weedayNumber.c

```
#include <stdio.h>

int main()
{
        int day;
        printf("Enter day number (1-7): ");
        scanf("%d", &day);
        switch(day) {
                case 1: printf("Sunday"); break;
                case 2: printf("Monday"); break;
                case 3: printf("Tuesday"); break;
                case 4: printf("Wednesday"); break;
                case 5: printf("Thursday"); break;
                case 6: printf("Friday"); break;
                case 7: printf("Saturday"); break;
                default: printf("Invalid day number"); break;
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter day number (1-7):
1
Sunday

Test Case - 2
User Output
Enter day number (1-7):
2
Monday

Test Case - 3
User Output

4
Wednesday

Test Case - 4
User Output
Enter day number (1-7):
8
Invalid day number

S.No: 20	Exp. Name: ***Simple Calculator Using Switch-Case***	**Date: 2026-09-30**

ID: 2602050330

2026-2030-Batch2-I1

Manipal University Jaipur

Aim:

Write a program to design a calculator that performs addition, subtraction, minus, and division operation. This program inputs two operands and an operator and then displays the calculated results.

Input Format:

- The program displays the prompt:

```
Enter first number:
```

- Enter the first number.
- The program then displays the prompt:

```
Enter operator (+, -, *, /):
```

- Enter one of the operators: `+`, `-`, `*`, `/`.
- The program then displays the prompt:

```
Enter second number:
```

- Enter the second number.

Output Format:

- For addition, subtraction, and multiplication, display the result using the format:

```
Result = <result>
```

- The result should be displayed up to 2 decimal places.
- For division by zero, display:

```
Division by zero is not possible
```

- For an invalid operator, display:

```
Invalid operator
```

Source Code:

calculator.c

```
#include <stdio.h>

int main()
{
        float a, b;
        char op;
        printf("Enter first number: ");
        scanf("%f", &a);
        printf("Enter operator (+, -, *, /): ");
        scanf(" %c", &op);
        printf("Enter second number: ");
        scanf("%f", &b);

        switch(op){
                case '+': printf("Result = %.2f", a+b); break;
                case '-': printf("Result = %.2f", a-b); break;
                case '*': printf("Result = %.2f", a*b); break;
                case '/':
                        if (b==0)
                                printf("Division by zero is not
possible");
                        else
                                printf("Result = %.2f", a/b);
                        break;
                default:
                        printf("Invalid operator"); break;
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter first number:
10
Enter operator (+, -, *, /):
+
Enter second number:
5

Test Case - 2
User Output
Enter first number:
20
Enter operator (+, -, *, /):
-
Enter second number:
8
Result = 12.00

Test Case - 3
User Output
Enter first number:
25
Enter operator (+, -, *, /):
/
Enter second number:
5
Result = 5.00

Test Case - 4
User Output
Enter first number:
10
Enter operator (+, -, *, /):
/
Enter second number:
0
Division by zero is not possible

Test Case - 5
User Output
Enter first number:
10

Enter second number:
5
Invalid operator

Test Case - 6
User Output
Enter first number:
3.5
Enter operator (+, -, *, /):
*
Enter second number:
8.2
Result = 28.70

S.No: 21	Exp. Name: ***Internet Browsing Bill Calculation Using Switch-Case***	**Date: 2026-09-30**

ID: 2602050330

Aim:

Write a program (using Switch case) to calculate a bill for internet browsing. The conditions are given below:

- Minimum Rs. 200 for up to 100 calls.
- Plus, Rs. 0.60 per call for the next 50 calls.
- Plus, Rs. 0.50 per call for the next 50 calls.
- Plus, Rs. 0.40 per call for any call beyond 200 calls.

Input Format:

- The program displays the prompt:

```
Enter number of calls:
```

- Enter an integer representing the number of calls.

Output Format:

- Display the calculated internet bill in the format:

```
Internet Bill = Rs. <bill>
```

- The bill should be displayed with two decimal places.

Source Code:

BillCalculation.c

```c
#include <stdio.h>

int main()
{
        int calls;
        float bill;
        printf("Enter number of calls: ");
        scanf("%d", &calls);

        switch(calls){
                case 1 ... 100:
                        bill=200;
                        break;
                case 101 ... 150:
                        bill=200+(calls-100)*0.60;
                        break;
                case 151 ... 200:
                        bill=230+(calls-150)*0.50;
                        break;
                default:
                        bill=255+(calls-200)*0.40;
                        break;
        }

        printf("Internet Bill = Rs. %.2f", bill);
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter number of calls:
50
Internet Bill = Rs. 200.00

Test Case - 2
User Output
Enter number of calls:

120
Internet Bill = Rs. 212.00

S.No: 22	Exp. Name: ***Telephone Bill Calculation Using Switch-Case***	**Date: 2026-09-30**

Aim:

Write a program (using Switch case) to calculate an amount of a telephone bill for the following criteria. (Without Loop)

Calls	**charge per call (Rs.)**
1-150	0
151-250	0.9
251-400	1.2
401 onwards	1.5

Input Format:

- The program displays the prompt:

```
Enter number of calls:
```

- Enter an integer representing the number of calls.

Output Format:

- Display the calculated telephone bill in the format:

```
Telephone Bill = Rs. <bill>
```

- The bill should be displayed with two decimal places.

Note: The bill must be calculated using progressive slab-wise billing, where each slab is charged only for the calls falling within that slab. For example, if the number of calls is 300, the first 150 calls are charged at Rs. 0, the next 100 calls at Rs. 0.9 per call, and the remaining 50 calls at Rs. 1.2 per call.

Source Code:

TelephoneBill.c

```
#include <stdio.h>

int main()
{
        int calls;
        float bill;
        printf("Enter number of calls: ");
        scanf("%d", &calls);

        switch(calls) {
                case 1 ... 150:
                        bill = 0;
                        break;
                case 151 ... 250:
                        bill = (calls - 150) * 0.90;
                        break;
                case 251 ... 400:
                        bill = 90 + (calls - 250) * 1.20;
                        break;
                default:
                        bill = 270 + (calls - 400) * 1.50;
                        break;
        }

        printf("Telephone Bill = Rs. %.2f", bill);
        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter number of calls:
100
Telephone Bill = Rs. 0.00

Test Case - 2
User Output

Enter number of calls:
200
Telephone Bill = Rs. 45.00

Test Case - 3
User Output
Enter number of calls:
250
Telephone Bill = Rs. 90.00

Test Case - 4
User Output
Enter number of calls:
399
Telephone Bill = Rs. 268.80

Test Case - 5
User Output
Enter number of calls:
402
Telephone Bill = Rs. 273.00

ID: 2602050330
2026-2030-Batch2-I1
Manipal University Jaipur

S.No: 23	Exp. Name: ***Electricity Bill Calculation Using Switch-Case***	**Date: 2026-09-30**

Aim:

Write a program (using Switch case) to calculate the amount of an electricity bill for the following criteria (Without Loop).

Units	**charge per unit (Rs.)**
First, 1-100	up to 0
Next, 101-200	up to 1.5
Next, 201-400	up to 2.5
401. onwards	3.5

Input Format:

- The program displays the prompt:

```
Enter number of units:
```

- Enter an integer representing the number of units consumed.

Output Format:

- Display the calculated electricity bill in the format:

```
Electricity Bill = Rs. <bill>
```

- The bill should be displayed with two decimal places.

Note: The bill must be calculated using progressive slab-wise billing, where each slab is charged only for the units falling within that slab. For example, if the consumption is 300 units, the first 100 units are charged at Rs. 0, the next 100 units at Rs. 1.5 per unit, and the remaining 100 units at Rs. 2.5 per unit.

Source Code:

ElectricityBill.c

```
#include <stdio.h>

int main()
{
        int units;
        float bill;
        printf("Enter number of units: ");
        scanf("%d", &units);

        switch(units){
                case 1 ... 100:
                        bill = 0;
                        break;
                case 101 ... 200:
                        bill = (units - 100) * 1.50;
                        break;
                case 201 ... 400:
                        bill = 150 + (units - 200) * 2.50;
                        break;
                default:
                        bill = 650 + (units - 400) * 3.50;
                        break;
        }

        printf("Electricity Bill = Rs. %.2f", bill);
        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter number of units:
50
Electricity Bill = Rs. 0.00

Test Case - 2
User Output

Enter number of units:
200
Electricity Bill = Rs. 150.00

Test Case - 3
User Output
Enter number of units:
400
Electricity Bill = Rs. 650.00

Test Case - 4
User Output
Enter number of units:
402
Electricity Bill = Rs. 657.00

S.No: 24	Exp. Name: ***Calculate Discount Based on Cost Price Using Switch-Case***	**Date: 2026-10-01**

Aim:

Write a program (using Switch case) to calculate the discount in rupees for the following criterion. (Without Loop)

Cost price	discount
>=800	25%
500-800	20%
<500	no discount (0%)

Input Format:

- The program displays the prompt:

```
Enter cost price:
```

- Enter a floating-point value representing the cost price.

Output Format:

- Display the calculated discount in the format:

```
Discount = Rs. <discount>
```

- The discount should be displayed with two decimal places.

Source Code:

DiscountPrice.c

```
#include <stdio.h>

int main()
{
        float price, discount;
        int choice;
        printf("Enter cost price: ");
        scanf("%f", &price);

        if (price>=800) choice = 1;
        else if (price>=500) choice = 2;
        else choice = 3;

        switch(choice){
                case 1:
                        discount = price * 25/100;
                        break;
                case 2:
                        discount = price * 20/100;
                        break;
                default:
                        discount = 0;
        }
        printf("Discount = Rs. %.2f", discount);
        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter cost price:
600
Discount = Rs. 120.00

Test Case - 2
User Output
Enter cost price:

499.99
Discount = Rs. 0.00

Test Case - 3
User Output
Enter cost price:
1200.50
Discount = Rs. 300.12

S.No: 25	Exp. Name: ***Check Whether a Number is Armstrong or Not***	**Date: 2026-10-01**

ID: 2602050330

2026-2030-Batch2-I1

Manipal University Jaipur

Aim:

Write a program to check whether the given number is Armstrong or not. The program should first determine the number of digits in the given number and calculate the sum of each digit raised to the power of the number of digits.

Input Format:

- The program displays the prompt "Enter a number: " followed by an integer representing a number to check whether it is an Armstrong number.

Output Format:

- The first line displays an integer representing the number of digits in the given number in the following format:

```
Length of Number = <length>
```

- The second line displays an integer representing the sum of each digit raised to the power of the number of digits in the following format:

```
Summation of Digits = <sum>
```

- If the given number is an Armstrong number, the third line displays:

```
Armstrong number
```

- Otherwise, displays:

```
Not an Armstrong number
```

Source Code:

ArmstrongNumber.c

```
#include <stdio.h>
int main()
{int n, temp, digits = 0, digit, power, i, sum=0;
printf("Enter a Number: ");
scanf("%d", &n);

temp = n;
do {
        digits++;
        temp=temp/10;
} while (temp>0);
printf("Length of Number = %d\n", digits);
temp = n;
while (temp > 0)
{
        digit = temp%10;
        power = 1;
        for (i=1; i<=digits; i++)
                power = power*digit;
        sum = sum + power;
        temp = temp/10;
}
printf("Summation of Digits = %d\n", sum);
if (sum==n)
        printf("Armstrong number");
 else
        printf("Not an Armstrong number");
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter a Number:
153
Length of Number = 3
Summation of Digits = 153
Armstrong number

Test Case - 2
User Output
Enter a Number:
370
Length of Number = 3
Summation of Digits = 370
Armstrong number

Test Case - 3
User Output
Enter a Number:
123
Length of Number = 3
Summation of Digits = 36
Not an Armstrong number

S.No: 26	Exp. Name: ***Sum of N Terms of the Sine Series***	**Date: 2026-10-01**

ID: 2602050330

2026-2030-Batch2-I1

Manipal University Jaipur

Aim:

Write a program to find the sum of n terms of the sin series
$\sin(x) = x - x^3 + x^5 - x^7 + \ldots$

Input Format:

- The program displays the prompt:

```
Enter x in radians:
```

- Enter a floating-point value x representing the angle in radians.
- The program then displays the prompt:

```
Enter number of terms:
```

- Enter an integer n representing the number of terms to be considered in the series.

Output Format:

- Display the sum of the first n terms of the sine series in the format:

```
Sum = <sum>
```

- The result should be displayed as a floating-point value.

Source Code:

SinSeries.c

```c
#include <stdio.h>

int main()
{
        float x;
        int n;
        printf("Enter x in radians: ");
        scanf("%f", &x);

        printf("Enter number of terms: ");
        scanf("%d", &n);

        float sum=0.0;
        float term = x;
        float x_squared = x*x;

        for (int i = 1; i<=n; i++){
                sum+=term;
                term=-term*x_squared/((2*i) * (2*i+1));
        }

        printf("Sum = %f", sum);
        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter x in radians:
1
Enter number of terms:
3
Sum = 0.841667

Test Case - 2
User Output

Manipal University Jaipur 2026-2030-Batch2-I1 ID: 2602050330 Page No: 68

```
0.5
Enter number of terms:
3
Sum = 0.479427
```

ID: 2602050330

2026-2030-Batch2-I1

Manipal University Jaipur

S.No: 27	Exp. Name: ***Reverse a Given Number***	Date: 2026-10-01

Aim:

Write a program to reverse a given number.

Example:

```
1234 reverse = 4 × 10³ + 3 × 10² + 2 × 10¹ + 1 × 10⁰ = 4321
```

Input Format:

- The program displays the prompt:

```
Enter a number:
```

- Enter an integer whose digits need to be reversed.

Output Format:

- Display the reversed number in the format:

```
Reverse = <reversed number>
```

Source Code:

ReverseNumber.c

```
#include <stdio.h>

int main()
{
        int n, reverse = 0, digit;
        printf("Enter a number: ");
        scanf("%d", &n);

        while (n>0)
                {
                        digit = n%10;
                        reverse = reverse * 10 + digit;
                        n = n/10;
                }
        printf("Reverse = %d\n", reverse);
        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1

User Output
Enter a number:
1234
Reverse = 4321

Test Case - 2
User Output
Enter a number:
56789
Reverse = 98765

S.No: 28	Exp. Name: ***Convert Binary Number to Decimal***	**Date: 2026-10-01**

Aim:

Write a program to convert the decimal number into binary to decimal.

Example:

```
1101 = 1 × 2³ + 1 × 2² + 0 × 2¹ + 1 × 2⁰ = 13
```

Input Format:

- The program displays the prompt:

```
Enter a binary number:
```

- Enter a binary number consisting of 0s and 1s.

Output Format:

- Display the equivalent decimal value in the format:

```
Decimal = <decimal value>
```

Source Code:

binaryConversion.c

```
#include <stdio.h>

int main()
{
        int binary, decimal=0, base=1, digit;
        printf("Enter a binary number: ");
        scanf("%d", &binary);

        while (binary > 0)
                {
                        digit = binary % 10;
                        decimal = decimal + digit * base;
                        base = base * 2;
                        binary = binary/10;
                }
        printf("Decimal = %d", decimal);
        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter a binary number:
1010
Decimal = 10

Test Case - 2
User Output
Enter a binary number:
1111
Decimal = 15

S.No: 29	Exp. Name: ***Check Whether a Number is Prime or Not***	**Date: 2026-10-01**

Aim:

Write a program to find whether the given number is prime or not.

Input Format:

- The program displays the prompt:

```
Enter a number:
```

- Enter an integer n to check whether it is a prime number.

Output Format:

- If the given number is prime, display:

```
Prime number
```

- If the given number is not prime, display:

```
Not a prime number
```

Source Code:

PrimeNumber.c

```
#include <stdio.h>

int main()
{
        int n, i, count=0;
        printf("Enter a number: ");
        scanf("%d", &n);

        for (int i = 1; i<=n; i++)
                {
                        if (n%i==0)
                                count++;
                }
        if (count == 2)
                        printf("Prime number");
        else
                printf("Not a prime number");
        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter a number:
7
Prime number

Test Case - 2
User Output
Enter a number:
10
Not a prime number

S.No: 30	Exp. Name: ***Find the Nth Armstrong Number***	**Date: 2026-10-01**

ID: 2602050330

2026-2030-Batch2-I1

Manipal University Jaipur

Aim:

Write a program to print the N^{th} Armstrong number.

Input Format:

- The program displays the prompt:

```
Enter the value of N:
```

- Enter an integer N, representing the position of the Armstrong number to be found.

Output Format:

- Display the N^{th} Armstrong number using the format:

```
The <N>th Armstrong number is <number>
```

Note: For this problem, 1 is considered the first Armstrong number.

Source Code:

ArmstrongNumber.c

```c
#include <stdio.h>

int main(){
        int n;
        printf("Enter the value of N: ");
        scanf("%d", &n);

        int count=0, num=1, ans=0;

        while (count < n){
                int temp=num;
                int digits=0;
                while (temp>0){
                        digits++;
                        temp/=10;
                }

                temp=num;
                int sum=0;
                while (temp>0){
                        int rem=temp%10;
                        int p=1;
                        for (int i=0; i<digits; i++){
                                p *= rem;
                        }
                        sum += p;
                        temp /= 10;
                }

                if (sum==num){
                        count++;
                        if (count == n){
                                ans=num;
                                break;
                        }
                }
                num++;
        }

        printf("The %dth Armstrong number is %d\n", n, ans);
        return 0;
}
```


ID: 2602050330
2026-2030-Batch2-I1
Manipal University Jaipur

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter the value of N:
1
The 1th Armstrong number is 1

Test Case - 2
User Output
Enter the value of N:
10
The 10th Armstrong number is 153

ID: 2602050330
2026-2030-Batch2-I1
Manipal University Jaipur

S.No: 31	Exp. Name: ***Print Prime Numbers up to N and Find Their Total Count***	**Date: 2026-10-01**

ID: 2602050330

2026-2030-Batch2-I1

Manipal University Jaipur

Aim:

Write a program to print the total count of prime numbers upto N and also print them.

Input Format:

- The program displays the prompt:

```
Enter N:
```

- Enter an integer N, representing the upper limit up to which the prime numbers should be found.

Output Format:

- For every prime number from 2 up to N, display the prime number using the format:

```
Prime Number is= <prime number>
```

- After printing all the prime numbers, display the total count using the format:

```
The total number of prime numbers are <count>
```

Source Code:

PrimeNumbers.c

```
#include <stdio.h>

int main()
{
        int n, count=0;
        printf("Enter N: ");
        scanf("%d", &n);

        for (int i=2; i<=n; i++){
                int isPrime = 1;
                for (int j=2; j*j<=i; j++){
                        if(i%j==0){
                                isPrime=0;
                                break;
                        }
                }
                if(isPrime){
                        printf("Prime Number is= %d\n", i);
                        count++;
                }
        }

        printf("The total number of prime numbers are %d\n", count);
        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter N:
10
Prime Number is= 2
Prime Number is= 3
Prime Number is= 5
Prime Number is= 7
The total number of prime numbers are 4

User Output
Enter N:
20
Prime Number is= 2
Prime Number is= 3
Prime Number is= 5
Prime Number is= 7
Prime Number is= 11
Prime Number is= 13
Prime Number is= 17
Prime Number is= 19
The total number of prime numbers are 8

S.No: 32	Exp. Name: ***Print Fibonacci Numbers Within a Given Range***	**Date: 2026-10-01**

Aim:

Write a program to print the Fibonacci numbers that fall in the given range.

Input Format:

Display the prompt:

```
Enter the range (start and end):
```

The program accepts two integers:

- $start$ — the starting value of the range.
- end — the ending value of the range.

Output Format:

- First, print the message:

```
Fibonacci numbers in the given range are:
```

- Then, print all the Fibonacci numbers that fall within the given range, separated by spaces.

Source Code:

Fibonaccinumbers.c

```
#include <stdio.h>

int main()
{
        int start, end, a=0, b=1, next;
        printf("Enter the range (start and end): ");
        scanf("%d%d", &start, &end);
        printf("Fibonacci numbers in the given range are:\n");
        while (a<=end){
                if(a>=start){
                        printf("%d ", a);
                }
                next=a+b;
                a=b;
                b=next;
        }

        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter the range (start and end):
1 20
Fibonacci numbers in the given range are:
1 1 2 3 5 8 13

Test Case - 2
User Output
Enter the range (start and end):
5 20
Fibonacci numbers in the given range are:
5 8 13

ID: 2602050330
2026-2030-Batch2-I1
Manipal University Jaipur

S.No: 33	Exp. Name: ***Generate Multiplication Tables for N Numbers up to K Terms Using Nested Loops***	**Date: 2026-10-01**

Aim:

Write a program to generate the multiplication table for n numbers up to $k4 terms (nested loops).

Input Format:

- The program first displays the prompt:

```
Enter the number of tables  upto n:
```

- Enter an integer n, representing the number of multiplication tables to generate.
- The program then displays the prompt:

```
Enter the number of terms (k):
```

- Enter an integer k, representing the number of terms in each multiplication table.

Output Format:

- For each number from 1 to n, display the corresponding multiplication table with the message:

```
Multiplication Table of <number>:
```

- Each multiplication result should be displayed in the format:

```
<number> x <term> = <result>
```

Source Code:

MultiplicationTable.c

```
#include <stdio.h>

int main()
{
        int n, k;
        printf("Enter the number of tables  upto n: ");
        scanf("%d", &n);

        printf("Enter the number of terms (k): ");
        scanf("%d", &k);

        for (int i=1; i<=n; i++) {
                printf("Multiplication Table of %d:\n", i);
                for (int j=1; j<=k; j++){
                        printf("%d x %d = %d\n", i, j, i*j);
                }
        }

        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter the number of tables upto n:
5
Enter the number of terms (k):
3
Multiplication Table of 1:
1 x 1 = 1
1 x 2 = 2
1 x 3 = 3
Multiplication Table of 2:
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
Multiplication Table of 3:
3 x 1 = 3

3 x 2 = 6
3 x 3 = 9
Multiplication Table of 4:
4 x 1 = 4
4 x 2 = 8
4 x 3 = 12
Multiplication Table of 5:
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15

Test Case - 2
User Output
Enter the number of tables upto n:
2
Enter the number of terms (k):
8
Multiplication Table of 1:
1 x 1 = 1
1 x 2 = 2
1 x 3 = 3
1 x 4 = 4
1 x 5 = 5
1 x 6 = 6
1 x 7 = 7
1 x 8 = 8
Multiplication Table of 2:
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
2 x 4 = 8
2 x 5 = 10
2 x 6 = 12
2 x 7 = 14
2 x 8 = 16

S.No: 34	Exp. Name: ***Print Different Patterns Using Nested Loops***	**Date: 2026-10-01**

ID: 2602050330

2026-2030-Batch2-I1

Manipal University Jaipur

Aim:

Write a program to print different patterns using nested loops.

The program should accept the number of rows n as input and print the following three patterns based on the given number of rows.

Pattern 1:

For n = 5:

```
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
```

Pattern 2:

For n = 5:

```
1
2 2
3 3 3
4 4 4 4
5 5 5 5 5
5 5 5 5 5
4 4 4 4
3 3 3
2 2
1
```

Pattern 3:

- Each row starts with the row number, and every subsequent number in the same row is increased by 2.

For n = 5:

```
1
2 4
3 5 7
4 6 8 10
5 7 9 11 13
```

The number of rows should be dynamic, so the patterns must be generated according to the value of n entered by the user.

Input Format:

- The program displays the prompt:

```
Enter number of rows:
```

- Enter an integer n, representing the number of rows to be printed for the patterns.

Output Format:

- Display the first pattern with the prompt:

```
Pattern 1:
```

- Display the second pattern with the prompt:

```
Pattern 2:
```

- The second pattern should first print rows from 1 to n and then print rows from n back to 1.
- Display the third pattern with the prompt:

```
Pattern-3:
```

- Numbers within each row should be separated by a space.

Source Code:

ID: 2602050330

Manipal University Jaipur

PatternPrinting.c

```c
#include <stdio.h>

int main()
{
        int n;
        printf("Enter number of rows: ");
        scanf("%d", &n);

        printf("Pattern 1:\n");
        int num=1;
        for (int i = 1; i <= n; i++){
                for (int j = 1; j<=i; j++){
                        printf("%d ", num++);
                }
                printf("\n");
        }

        printf("Pattern 2:\n");
        for (int i = 1; i<=n; i++){
                for (int j = 1; j <= i; j++){
                        printf("%d ", i);
                }
                printf("\n");
        }
        for (int i = n; i >= 1; i--){
                for (int j = 1; j<=i; j++){
                        printf("%d ", i);
                }
                printf("\n");
        }

        printf("Pattern-3:\n");
        for (int i =1; i<=n; i++){
                int val = i;
                for (int j = 1; j<=i; j++){
                        printf("%d ", val);
                        val+=2;
                }
                printf("\n");
        }
}
```

Manipal University Jaipur 2026-2030-Batch2-I1 ID: 2602050330 Page No: 89

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter number of rows:
5
Pattern 1:
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
Pattern 2:
1
2 2
3 3 3
4 4 4 4
5 5 5 5 5
5 5 5 5 5
4 4 4 4
3 3 3
2 2
1
Pattern-3:
1
2 4
3 5 7
4 6 8 10
5 7 9 11 13

Test Case - 2
User Output
Enter number of rows:
6
Pattern 1:
1
2 3
4 5 6
7 8 9 10

Pattern 2:
1
2 2
3 3 3
4 4 4 4
5 5 5 5 5
6 6 6 6 6 6
6 6 6 6 6 6
5 5 5 5 5
4 4 4 4
3 3 3
2 2
1
Pattern-3:
1
2 4
3 5 7
4 6 8 10
5 7 9 11 13
6 8 10 12 14 16

S.No: 35	Exp. Name: ***Finding the Largest and Smallest Element in an Array***	**Date: 2026-10-07**

ID: 2602050330

2026-2030-Batch2-I1

Manipal University Jaipur

Aim:

Write a program to find the largest and smallest element in an array.

Input Format:

- The program should display the prompt:

```
Enter number of elements:
```

- The user should enter a single integer representing the number of elements, n, on the same line as the prompt.
- The program should then display the prompt:

```
Enter n elements:
```

- The user should enter n integers, separated by spaces, on the next line.

Output Format:

- The program should display the largest element in the array in the format:

```
Largest element = <largest>
```

- The program should display the smallest element in the array in the format:

```
Smallest element = <smallest>
```

Source Code:

smallestLargest.c

```
#include <stdio.h>

int main()
{
        int n, arr[100];
        printf("Enter number of elements: ");
        scanf("%d", &n);

        printf("Enter %d elements:\n", n);
        for (int i = 0; i < n; i++)
                scanf("%d", &arr[i]);

        int largest = arr[0];
        int smallest = arr[0];

        for (int i = 1; i < n; i++){
                if (arr[i] > largest) {
                        largest = arr[i];
                }
                if (arr[i] < smallest) {
                        smallest = arr[i];
                }
        }

        printf("Largest element = %d\n", largest);
        printf("Smallest element = %d\n", smallest);

        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter number of elements:
5
Enter 5 elements:
3 7 1 9 4
Largest element = 9

Test Case - 2
User Output
Enter number of elements:
4
Enter 4 elements:
-5 -1 -10 -3
Largest element = -1
Smallest element = -10

S.No: 36	Exp. Name: ***Sum of Odd Index Numbers in an Array***	**Date: 2026-10-07**

ID: 2602050330

2026-2030-Batch2-I1

Manipal University Jaipur

Aim:

Write a program to find the sum of odd index numbers in an array.

Input Format:

- The program should display the prompt:

```
Enter number of elements:
```

- The user should enter a single integer representing the number of elements, n, on the same line as the prompt.
- The program should then display the prompt:

```
Enter array elements:
```

- The user should enter n integers, separated by spaces, on the next line after the prompt.

Output Format:

- The program should display the sum of the elements present at odd indexes (index 1, 3, 5, ...) of the array in the format:

```
Sum of elements at odd indexes = <sum>
```

Source Code:

oddIndexSum.c

```c
#include <stdio.h>

int main()
{
        int n, arr[100], sum=0;
        printf("Enter number of elements: ");
        scanf("%d", &n);

        printf("Enter array elements:\n");
        for (int i = 0; i < n; i++)
                scanf("%d", &arr[i]);

        for (int i = 0; i < n; i++)
        {
                if (i % 2 != 0){
                        sum+=arr[i];
                }
        }

        printf("Sum of elements at odd indexes = %d", sum);
        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter number of elements:
5
Enter array elements:
1 2 3 4 5
Sum of elements at odd indexes = 6

Test Case - 2
User Output
Enter number of elements:
6

```
-2 5 -4 8 -6 3
```

```
Sum of elements at odd indexes = 16
```

S.No: 37	Exp. Name: ***Printing the Subarray Between Two Indexes***	**Date: 2026-10-07**

ID: 2602050330

2026-2030-Batch2-I1

Manipal University Jaipur

Aim:

Write a program to print the subarray that lies between the two indexes.

Input Format:

- The program should display the prompt:

```
Enter number of elements:
```

- The user should enter a single integer representing the number of elements, n, on the same line as the prompt.
- The program should then display the prompt:

```
Enter array elements:
```

- The user should enter n integers, separated by spaces, on the next line after the prompt.
- The program should then display the prompt:

```
Enter two indexes:
```

- The user should enter two integers representing the **start** and **end** indexes, separated by a space, on the same line as the prompt.

Output Format:

- If the indexes are valid (i.e., **start** ≥ 0, **end** $< n$, and **start** $\leq$ **end**), the program should display:

```
Subarray is:
```

- followed by the elements of the array from **start** index to **end** index (inclusive), each separated by a space.
- If the indexes are invalid, the program should display:

```
Invalid indexes
```

Source Code:

subArray.c

```
#include <stdio.h>

int main()
{
        int n, arr[100], start, end;
        printf("Enter number of elements: ");
        scanf("%d", &n);

        printf("Enter array elements:\n");
        for (int i = 0; i < n; i++)
                scanf("%d", &arr[i]);

        printf("Enter two indexes: ");
        scanf("%d%d", &start, &end);
        if(start >= 0 && end < n && start <= end){
                printf("Subarray is:\n");
                for (int i = start; i <= end; i++){
                        printf("%d ", arr[i]);
                }
                printf("\n");
        } else {
                printf("Invalid indexes\n");
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter number of elements:
5
Enter array elements:
10 20 30 40 50
Enter two indexes:
1 3
Subarray is:
20 30 40

Test Case - 2
User Output
Enter number of elements:
3
Enter array elements:
5 6 7
Enter two indexes:
2 1
Invalid indexes

S.No: 38	Exp. Name: ***Counting Positive, Negative, Odd, Even, and Zero Numbers in an Array***	**Date: 2026-10-07**

ID: 2602050330

2026-2030-Batch2-I1

Manipal University Jaipur

Aim:

Write a program to find the number of positive numbers, negative numbers, odd numbers, even numbers, and the number of 0 of an array.

Input Format:

- The program should display the prompt:

```
Enter number of elements:
```

- The user should enter a single integer representing the number of elements,n, on the same line as the prompt.
- The program should then display the prompt:

```
Enter array elements:
```

- The user should enter$n$ integers, separated by spaces, on the next line (after the prompt).

Output Format:

- The program should display the count of positive numbers in the format:

```
Positive numbers = <count>
```

- The program should display the count of negative numbers in the format:

```
Negative numbers = <count>
```

- The program should display the count of odd numbers (excluding 0) in the format:

```
Odd numbers = <count>
```

- The program should display the count of even numbers (excluding 0) in the format:

```
Even numbers = <count>
```

- The program should display the count of zeros in the format:

```
Zero numbers = <count>
```

Source Code:

numberType.c

```c
#include <stdio.h>

int main()
{
        int n, arr[100], pos=0, neg=0, odd=0, even=0, zero=0;
        printf("Enter number of elements: ");
        scanf("%d", &n);

        printf("Enter array elements:\n");
        for (int i = 0; i < n; i++){
                scanf("%d", &arr[i]);
        }

        for (int i = 0;  i< n; i++){
                if (arr[i] > 0){
                        pos++;
                }
                else if (arr[i] < 0){
                        neg++;
                }
                else {
                        zero++;
                }

                if (arr[i] != 0){
                        if (arr[i] % 2 == 0){
                                even++;
                        }
                        else {
                                odd++;
                        }
                }
        }

        printf("Positive numbers = %d\n", pos);
        printf("Negative numbers = %d\n", neg);
        printf("Odd numbers = %d\n", odd);
        printf("Even numbers = %d\n", even);
        printf("Zero numbers = %d\n", zero);

        return 0;
}
```

Manipal University Jaipur 2026-2030-Batch2-I1 ID: 2602050330 Page No: 102

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter number of elements:
6
Enter array elements:
3 -2 0 5 -8 0
Positive numbers = 2
Negative numbers = 2
Odd numbers = 2
Even numbers = 2
Zero numbers = 2

Test Case - 2
User Output
Enter number of elements:
5
Enter array elements:
-2 -4 -6 -8 -10
Positive numbers = 0
Negative numbers = 5
Odd numbers = 0
Even numbers = 5
Zero numbers = 0

S.No: 39	Exp. Name: ***Reversing an Array Using an Auxiliary Array***	**Date: 2026-10-07**

ID: 2602050330

2026-2030-Batch2-I1

Manipal University Jaipur

Aim:

Write a program to reverse an array with an auxiliary array.

Input Format:

- The program should display the prompt:

```
Enter number of elements:
```

- The user should enter a single integer representing the number of elements, n, on the same line as the prompt.
- The program should then display the prompt:

```
Enter array elements:
```

- The user should enter n integers, separated by spaces, on the next line after the prompt.

Output Format:

- The program should display the message:

```
Reversed array:
```

- followed by the elements of the reversed array (stored in the auxiliary array), each separated by a space.

Source Code:

reverseArray.c

```
#include <stdio.h>

int main()
{
        int arr[100], arr2[100], n;
        printf("Enter number of elements: ");
        scanf("%d", &n);

        printf("Enter array elements:\n");
        for (int i = 0; i < n; i++)
                scanf("%d", &arr[i]);

        for (int i = 0; i < n; i++)
                arr2[i] = arr[n-1-i];

        printf("Reversed array:\n");
        for (int i = 0; i < n; i++)
                printf("%d ", arr2[i]);
        printf("\n");

        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter number of elements:
5
Enter array elements:
1 2 3 4 5
Reversed array:
5 4 3 2 1

Test Case - 2
User Output
Enter number of elements:

Enter array elements:
-1 -2 -3 -4
Reversed array:
-4 -3 -2 -1

S.No: 40	Exp. Name: ***Checking Whether an Array is Sorted***	**Date: 2026-10-07**

Aim:

Write a program to check whether an array is sorted or not.

Input Format:

- The program should display the prompt:

```
Enter number of elements:
```

- The user should enter a single integer representing the number of elements, n, on the same line as the prompt.
- The program should then display the prompt:

```
Enter array elements:
```

- The user should enter n integers, separated by spaces, on the next line after the prompt.

Output Format:

- If the array is sorted in ascending order, the program should display:

```
Array is sorted in ascending order
```

- Otherwise, the program should display:

```
Array is not sorted
```

Source Code:

ID: 2602050330 Page No: 107
2026-2030-Batch2-I1 Manipal University Jaipur

arrayOrder.c

```
#include <stdio.h>

int main()
{
        int n, arr[100], sorted=1;
        printf("Enter number of elements: ");
        scanf("%d", &n);

        printf("Enter array elements:\n");
        for (int i = 0; i < n; i++)
                scanf("%d", &arr[i]);

        for (int i = 0; i < n-1; i++){
                if (arr[i] > arr[i+1]){
                        sorted=0;
                        break;
                }
        }

        if (sorted){
                printf("Array is sorted in ascending order\n");
        } else {
                printf("Array is not sorted\n");
        }

        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter number of elements:
5
Enter array elements:
1 2 3 4 5
Array is sorted in ascending order

User Output
Enter number of elements:
4
Enter array elements:
4 3 2 1
Array is not sorted

S.No: 41	Exp. Name: ***Sorting an Array in Ascending Order***	**Date: 2026-10-07**

ID: 2602050330

2026-2030-Batch2-I1

Manipal University Jaipur

Aim:

Write a program to arrange the elements of an array in ascending order by a simple sorting method. (Selection sort/bubble sort)

Input Format:

- The program should display the prompt:

```
Enter number of elements:
```

- The user should enter a single integer representing the number of elements, n, on the same line as the prompt.
- The program should then display the prompt:

```
Enter array elements:
```

- The user should enter n integers, separated by spaces, on the next line after the prompt.

Output Format:

- The program should display the message:

```
Array in ascending order:
```

- followed by the sorted array elements, each separated by a space.

Source Code:

sortArrayAscending.c

```
#include <stdio.h>

int main()
{
        int n, temp, a[100];
        printf("Enter number of elements: ");
        scanf("%d", &n);

        printf("Enter array elements:\n");
        for (int i = 0; i < n; i++)
                scanf("%d", &a[i]);

        for (int i = 0; i < n - 1; i++){
                for (int j = 0; j < n - i - 1; j++){
                        if (a[j] > a[j+1]) {
                                temp = a[j];
                                a[j] = a[j+1];
                                a[j+1] = temp;
                        }
                }
        }

        printf("Array in ascending order:\n");
        for (int i = 0; i < n; i++){
                printf("%d ", a[i]);
        }
        printf("\n");

        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter number of elements:
5
Enter array elements:
5 3 1 4 2
Array in ascending order:

1 2 3 4 5

Test Case - 2
User Output
Enter number of elements:
6
Enter array elements:
-3 5 -1 0 8 -2
Array in ascending order:
-3 -2 -1 0 5 8

S.No: 42	Exp. Name: ***Splitting an Array into Two Halves***	**Date: 2026-10-07**

0330

Aim:

Write a program to take an array of 10 elements. Split it into the middle and store the elements in two different arrays.

E.g.- Initial array:

58	24	13	15	63	9	8	81	1	78

After splitting:

58	24	13	15	63

9	8	81	1	78

(The program should split a fixed-size array of 10 elements into two halves of 5 elements each — the first 5 elements stored in one array and the last 5 elements stored in another.)

Input Format:

- The program should display the prompt:

```
Enter 10 elements:
```

- The user should enter 10 integers, separated by spaces, on the next line after the prompt.

Output Format:

- The program should display the message:

```
First array:
```

- followed by the first 5 elements, each separated by a space.
- The program should display the message:

```
Second array:
```

- followed by the last 5 elements, each separated by a space.

Source Code:

Manipal University Jaipur

splitArray.c

```c
#include <stdio.h>

int main()
{
        int arr[10], arr1[5], arr2[5];

        printf("Enter 10 elements:\n");
        for (int i = 0; i < 10; i++)
                scanf("%d", &arr[i]);

        for (int i = 0; i < 5; i++){
                arr1[i] = arr[i];
                arr2[i] = arr[i+5];
        }

        printf("First array:\n");
        for (int i = 0; i < 5; i++){
                printf("%d ", arr1[i]);
        }
        printf("\n");

        printf("Second array:\n");
        for (int i = 0; i < 5; i++){
                printf("%d ", arr2[i]);
        }
        printf("\n");

        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter 10 elements:
1 2 3 4 5 6 7 8 9 10
First array:
1 2 3 4 5
Second array:
6 7 8 9 10

Manipal University Jaipur 2026-2030-Batch2-I1 ID: 2602050330 Page No: 114

Test Case - 2
User Output
Enter 10 elements:
-5 -4 -3 -2 -1 1 2 3 4 5
First array:
-5 -4 -3 -2 -1
Second array:
1 2 3 4 5

S.No: 43	Exp. Name: ***Total Marks of Each Student and Average Marks of Each Subject***	**Date: 2026-10-15**

ID: 2602050330

2026-2030-Batch2-I1

Manipal University Jaipur

Aim:

Write a program to take 3 student marks in 5 subjects. Print the total marks of each student and the average marks of each subject.

Input Format:

- The program should display the prompt:

`Enter marks of 3 students in 5 subjects:`

- For each of the 3 students (i = 1 to 3), the program should display the prompt:

`Enter marks for Student <i>:`

- For each of the 5 subjects (j = 1 to 5) of that student, the program should display the prompt:

`Subject <j>:`

- The user should enter a single integer representing the marks for that subject, on the same line as the prompt.

Output Format:

- The program should display the message:

`Total Marks of Each Student:`

- For each student, the program should display the total marks in the format:

`Student <i> = <total>`

- The program should display the message:

`Average Marks of Each Subject:`

- For each subject, the program should display the average marks (rounded to 2 decimal places) in the format:

`Subject <j> = <average>`

Source Code:

studentmarks.c

```c
#include <stdio.h>

int main()
{
        int marks[3][5];
        printf("Enter marks of 3 students in 5 subjects:\n");
        for (int i = 0; i < 3; i++) {
                printf("Enter marks for Student %d:\n", i+1);
                for (int j = 0; j < 5; j++) {
                        printf("Subject %d: ", j+1);
                        scanf("%d", &marks[i][j]);
                }
        }

        printf("Total Marks of Each Student:\n");
        for (int i = 0; i < 3; i++) {
                int total = 0;
                for (int j = 0; j < 5; j++) {
                        total += marks[i][j];
                }
                printf("Student %d = %d\n", i+1, total);
        }

        printf("Average Marks of Each Subject:\n");
        for (int j = 0; j < 5; j++) {
                int sum = 0;
                for (int i = 0; i < 3; i++) {
                        sum += marks[i][j];
                }
                float average = (float)sum/3.0;
                printf("Subject %d = %.2f\n", j+1, average);
        }

        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output

```
Enter marks of 3 students in 5 subjects:
Enter marks for Student 1:
Subject 1:
80
Subject 2:
70
Subject 3:
90
Subject 4:
60
Subject 5:
50
Enter marks for Student 2:
Subject 1:
75
Subject 2:
85
Subject 3:
95
Subject 4:
65
Subject 5:
55
Enter marks for Student 3:
Subject 1:
100
Subject 2:
90
Subject 3:
80
Subject 4:
70
Subject 5:
60
Total Marks of Each Student:
Student 1 = 350
Student 2 = 375
Student 3 = 400
Average Marks of Each Subject:
Subject 1 = 85.00
```

Subject 4 = 65.00
Subject 5 = 55.00

Test Case - 2
User Output
Enter marks of 3 students in 5 subjects:
Enter marks for Student 1:
Subject 1:
100
Subject 2:
100
Subject 3:
100
Subject 4:
100
Subject 5:
100
Enter marks for Student 2:
Subject 1:
98
Subject 2:
98
Subject 3:
97
Subject 4:
96
Subject 5:
95
Enter marks for Student 3:
Subject 1:
75
Subject 2:
76
Subject 3:
77
Subject 4:
78
Subject 5:

```
Student 1 = 500
Student 2 = 484
Student 3 = 385
Average Marks of Each Subject:
Subject 1 = 91.00
Subject 2 = 91.33
Subject 3 = 91.33
Subject 4 = 91.33
Subject 5 = 91.33
```

S.No: 44	Exp. Name: ***Searching for an Element in a Matrix and Counting Occurrences***	**Date: 2026-10-15**

Aim:

Write a program for searching for an element in the matrix and counting the number of occurrences of that element.

Input Format:

- The program should display the prompt:

```
Enter number of rows and columns:
```

- The user should enter two integers representing the number of rows and columns, separated by a space, on the same line as the prompt.
- The program should then display the prompt:

```
Enter matrix elements:
```

- The user should enter rows × cols integers representing the matrix elements, row by row, separated by spaces, on the next line after the prompt.
- The program should then display the prompt:

```
Enter element to search:
```

- The user should enter a single integer representing the element to search for, on the same line as the prompt.

Output Format:

- If the element is found (count > 0), the program should display:

```
<element> found <count> time(s)
```

- If the element is not found, the program should display:

```
<element> not found in the matrix
```

Source Code:

elemSearch.c

```
#include <stdio.h>

int main()
{
        int r, c, s, count=0;
        printf("Enter number of rows and columns: ");
        scanf("%d%d", &r, &c);

        int matrix[r][c];
        printf("Enter matrix elements:\n");
        for (int i=0; i<r; i++) {
                for (int j=0; j<c; j++) {
                        scanf("%d", &matrix[i][j]);
                }
        }

        printf("Enter element to search: ");
        scanf("%d", &s);

        for (int i=0; i<r; i++) {
                for (int j=0; j<c; j++) {
                        if(matrix[i][j] == s) {
                                count++;
                        }
                }
        }

        if (count>0) {
                printf("%d found %d time(s)\n", s, count);
        }
        else {
                printf("%d not found in the matrix\n", s);
        }

        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1

Enter number of rows and columns:
2 3
Enter matrix elements:
1 2 3
4 2 5
Enter element to search:
2
2 found 2 time(s)

Test Case - 2
User Output
Enter number of rows and columns:
3 3
Enter matrix elements:
1 2 3
4 5 6
7 8 9
Enter element to search:
10
10 not found in the matrix

S.No: 45	Exp. Name: ***Multiplying Two Matrices***	Date: 2026-10-15

Aim:

Write a program to multiply two matrices.

Input Format:

- The program should display the prompt:

```
Enter rows and columns of first matrix:
```

- The user should enter two integers representing the rows and columns of the first matrix, separated by a space, on the same line as the prompt.
- The program should then display the prompt:

```
Enter rows and columns of second matrix:
```

- The user should enter two integers representing the rows and columns of the second matrix, separated by a space, on the same line as the prompt.
- The program should then display the prompt:

```
Enter first matrix:
```

- The user should enter $r1 \times c1$ integers representing the elements of the first matrix, row by row, separated by spaces, on the next line after the prompt.
- The program should then display the prompt:

```
Enter second matrix:
```

- The user should enter $r2 \times c2$ integers representing the elements of the second matrix, row by row, separated by spaces, on the next line after the prompt.

Output Format:

- If the number of columns of the first matrix is not equal to the number of rows of the second matrix, the program should display:

```
Matrix multiplication is not possible
```

- Otherwise, the program should display the message:

```
Resultant Matrix:
```

- followed by the resultant matrix, with each row's elements separated by a tab, and each row on a new line.

Source Code:

matrixMultiplication.c

```
#include <stdio.h>

int main()
{
        int r1, c1, r2, c2;
        printf("Enter rows and columns of first matrix: ");
        scanf("%d%d", &r1, &c1);
        printf("Enter rows and columns of second matrix: ");
        scanf("%d%d", &r2, &c2);

        int mat1[r1][c1], mat2[r2][c2], res[r1][c2];

        if (c1!=r2) {
                printf("Matrix multiplication is not possible\n");
        } else {
        printf("Enter first matrix:\n");
        for (int i=0; i<r1; i++) {
                for (int j=0; j<c1; j++) {
                        scanf("%d", &mat1[i][j]);
                }
        }
        printf("Enter second matrix:\n");
        for (int i=0; i<r2; i++) {
                for (int j=0; j<c2; j++) {
                        scanf("%d", &mat2[i][j]);
                }
        }

                printf("Resultant Matrix:\n");
                for (int i=0; i<r1; i++) {
                        for (int j=0; j<c2; j++) {
                                res[i][j] = 0;
                                for (int k=0; k<c1; k++) {
                                        res[i][j] += mat1[i][k] *
mat2[k][j];
                                }
                                printf("%d\t", res[i][j]);
                        }
                        printf("\n");
                }
        }

        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter rows and columns of first matrix:
2 3
Enter rows and columns of second matrix:
2 2
Matrix multiplication is not possible

Test Case - 2
User Output
Enter rows and columns of first matrix:
2 2
Enter rows and columns of second matrix:
2 2
Enter first matrix:
1 2
3 4
Enter second matrix:
5 6
7 8
Resultant Matrix:
19 22
43 50

S.No: 46	Exp. Name: ***Checking if a Matrix is a Magic Square***	**Date: 2026-10-15**

ID: 2602050330
2026-2030-Batch2-I1
Manipal University Jaipur

Aim:

Write a program to check if the given matrix is a magic square or not

A magic square has equal sums for all rows, columns, and both diagonals.

Input Format:

- The program should display the prompt:

```
Enter size of square matrix:
```

- The user should enter a single integer representing the size of the square matrix, n, on the same line as the prompt.
- The program should then display the prompt:

```
Enter matrix elements:
```

- The user should enter $n \times n$ integers representing the matrix elements, row by row, separated by spaces, on the next line after the prompt.

Output Format:

- If the matrix is a magic square, the program should display:

```
The matrix is a Magic Square
```

- Otherwise, the program should display:

```
The matrix is NOT a Magic Square
```

Source Code:

magicSquare.c

```
#include <stdio.h>

int main()
{
        int n;
        printf("Enter size of square matrix: ");
        scanf("%d", &n);

        int mat[n][n];
        printf("Enter matrix elements:\n");
        for (int i=0; i<n; i++) {
                for (int j=0; j<n; j++) {
                        scanf("%d", &mat[i][j]);
                }
        }

        int d1=0, d2=0;
        for (int i=0; i<n; i++) {
                d1+=mat[i][i];
                d2+=mat[i][n-1-i];
        }

        int isMagic=1;
        if (d1!=d2) {
                isMagic=0;
        } else {
                for (int i=0; i<n; i++) {
                        int rowSum=0, colSum=0;
                        for (int j=0; j<n; j++) {
                                rowSum += mat[i][j];
                                colSum += mat[j][i];
                        }
                        if (rowSum != d1 || colSum != d1) {
                                isMagic=0;
                                break;
                        }
                }
        }

        if (isMagic) {
                printf("The matrix is a Magic Square\n");
        } else {
                printf("The matrix is NOT a Magic Square\n");
        }
```

```
        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter size of square matrix:
3
Enter matrix elements:
2 7 6
9 5 1
4 3 8
The matrix is a Magic Square

Test Case - 2
User Output
Enter size of square matrix:
3
Enter matrix elements:
1 2 3
4 5 6
7 8 9
The matrix is NOT a Magic Square

S.No: 47	Exp. Name: ***Checking if a Matrix is Symmetric***	**Date: 2026-10-15**

ID: 2602050330

2026-2030-Batch2-I1

Manipal University Jaipur

Aim:

Write a program to find whether a given matrix is symmetric or not.

A matrix is symmetric if: A = A^T, meaning a[i][j] = a[j][i]

Input Format:

The program should display the prompt:

```
Enter size of square matrix:
```

- The user should enter a single integer representing the size of the square matrix, n, on the same line as the prompt.
- The program should then display the prompt:

```
Enter matrix elements:
```

- The user should enter $n \times n$ integers representing the matrix elements, row by row, separated by spaces, on the next line after the prompt.

Output Format:

- If the matrix is symmetric, the program should display:

```
The matrix is Symmetric
```

- Otherwise, the program should display:

```
The matrix is NOT Symmetric
```

Source Code:

symmetricmatrix.c

```
#include <stdio.h>

int main()
{
        int n;
        printf("Enter size of square matrix: ");
        scanf("%d", &n);

        int a[n][n];

        printf("Enter matrix elements:\n");
        for (int i=0; i<n; i++) {
                for (int j=0; j<n; j++) {
                        scanf("%d", &a[i][j]);
                }
        }

        int sym=1;
        for (int i=0; i<n; i++) {
                for (int j=0; j<n; j++) {
                        if (a[i][j] != a[j][i]) {
                                sym=0;
                                break;
                        }
                }
                if (!sym) {
                        break;
                }
        }

        if (sym) {
                printf("The matrix is Symmetric\n");
        } else {
                printf("The matrix is NOT Symmetric\n");
        }

        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter size of square matrix:
3
Enter matrix elements:
1 2 3
2 4 3
3 5 6
The matrix is NOT Symmetric

Test Case - 2
User Output
Enter size of square matrix:
3
Enter matrix elements:
1 2 3
2 4 5
3 5 6
The matrix is Symmetric

S.No: 48	Exp. Name: ***Finding the Trace and Norm of a Square Matrix***	**Date: 2026-10-15**

Aim:

Write a program to find the trace and norm of a given square matrix.

Hint: Trace = sum of principal diagonal elements
Norm = sqrt (sum of squares of the individual elements of an array)

Input Format:

- The program should display the prompt:

```
Enter size of square matrix:
```

- The user should enter a single integer representing the size of the square matrix, n, on the same line as the prompt.
- The program should then display the prompt:

```
Enter matrix elements:
```

- The user should enter $n \times n$ integers representing the matrix elements, row by row, separated by spaces, on the next line after the prompt.

Output Format:

- The program should display the trace of the matrix in the format:

```
Trace of matrix = <trace>
```

- The program should display the norm of the matrix (rounded to 2 decimal places) in the format:

```
Norm of matrix = <norm>
```

Source Code:

traceandNorm.c

```c
#include <stdio.h>
#include <math.h>

int main()
{
        int n;
        printf("Enter size of square matrix: ");
        scanf("%d", &n);

        int a[n][n], trace=0, squareSum=0;
        printf("Enter matrix elements:\n");
        for (int i=0; i<n; i++) {
                for (int j=0; j<n; j++) {
                        scanf("%d", &a[i][j]);

                        if (i==j) {
                                trace += a[i][j];
                        }

                        squareSum += a[i][j] * a[i][j];
                }
        }

        double norm = sqrt(squareSum);
        printf("Trace of matrix = %d\n", trace);
        printf("Norm of matrix = %.2f\n", norm);

        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter size of square matrix:
2
Enter matrix elements:
1 2
3 4

Manipal University Jaipur 2026-2030-Batch2-I1 ID: 2602050330 Page No: 134

Norm of matrix = 5.48

Test Case - 2
User Output
Enter size of square matrix:
3
Enter matrix elements:
1 0 0
0 1 0
0 0 1
Trace of matrix = 3
Norm of matrix = 1.73

S.No: 49	Exp. Name: ***Converting Lowercase Letters to Uppercase in a Sentence***	**Date: 2026-10-15**

Aim:

Write a program to change all lower-case letters into upper case in a sentence.

Input Format:

- The program should display the prompt:

```
Enter a sentence:
```

- The user should enter a sentence (a string, possibly containing spaces), on the same line as the prompt.

Output Format:

- The program should display the modified sentence in the format:

```
Modified sentence with all lowercase letters converted to uppercase: <sentence>
```

Source Code:

caseConversion.c

```
#include <stdio.h>

int main()
{
        char str[100];
        printf("Enter a sentence: ");
        scanf("%[^\n]", str);

        for (int i=0; str[i]; i++) {
                if (str[i] >= 'a' && str[i] <= 'z') {
                        str[i] = str[i] - 32;
                }
        }

        printf("Modified sentence with all lowercase letters converted
to uppercase: %s\n", str);
        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1

User Output
Enter a sentence:
hello world
Modified sentence with all lowercase letters converted to uppercase: HELLO WORLD

Test Case - 2
User Output
Enter a sentence:
already UPPER
Modified sentence with all lowercase letters converted to uppercase: ALREADY UPPER

S.No: 50	Exp. Name: ***Finding the Last Occurrence of a Character in a String***	**Date: 2026-10-15**

Aim:

Write a program to find the last occurrence of a particular character.

Input Format:

- The program should display the prompt:

```
Enter a string:
```

- The user should enter a string (possibly containing spaces), on the same line as the prompt.
- The program should then display the prompt:

```
Enter the target character:
```

- The user should enter a single character representing the target character to search for, on the same line as the prompt.

Output Format:

- If the target character is found in the string, the program should display:

```
Last occurrence of '<target>' is at index <last_occurrence>
```

- If the target character is not found, the program should display:

```
'<target>' not found in the string
```

Source Code:

lastOccurrence.c

```
#include <stdio.h>

int main()
{
        char str[100], target;
        printf("Enter a string: ");
        scanf("%[^\n]", str);

        printf("Enter the target character: ");
        scanf(" %c", &target);

        int lastIndex = -1;

        for (int i=0; str[i]; i++) {
                if (str[i] == target) {
                        lastIndex = i;
                }
        }

        if (lastIndex != -1) {
                printf("Last occurrence of '%c' is at index %d\n",
target, lastIndex);
        } else {
                printf("'%c' not found in the string\n", target);
        }
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter a string:
hello world
Enter the target character:
o
Last occurrence of 'o' is at index 7

Test Case - 2

User Output
Enter a string:
aaaa
Enter the target character:
a
Last occurrence of 'a' is at index 3

Test Case - 3
User Output
Enter a string:
C Programming
Enter the target character:
x
'x' not found in the string

S.No: 51	Exp. Name: ***Concatenating, Finding Length, and Copying Strings Using Library Functions***	**Date: 2026-10-15**

Aim:

Write a program to concatenate/length/copy two strings using the library function.

Input Format:

- The program should display the prompt:

```
Enter the first string:
```

- The user should enter the first string, on the same line as the prompt.
- The program should then display the prompt:

```
Enter the second string:
```

- The user should enter the second string, on the same line as the prompt.

Output Format:

- The program should display the concatenated string in the format:

```
Concatenated string: <concat_str>
```

- The program should display the length of the concatenated string in the format:

```
Length of concatenated string: <length>
```

- The program should display the copied string in the format:

```
Copied string: <copied_str>
```

Source Code:

stringFunctions.c

```c
#include <stdio.h>
#include <string.h>

int main() {
        char s1[100], s2[100], concat[200], copy[100];

        printf("Enter the first string: ");
        scanf(" %[^\n]", s1);

        printf("Enter the second string: ");
        scanf(" %[^\n]", s2);

        strcpy(concat, s1);
        strcat(concat, s2);
        strcpy(copy, concat);

        int len = strlen(concat);

        printf("Concatenated string: %s\n", concat);
        printf("Length of concatenated string: %d\n", len);
        printf("Copied string: %s\n", copy);

        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter the first string:
C
Enter the second string:
Programming
Concatenated string: CProgramming
Length of concatenated string: 12
Copied string: CProgramming

Test Case - 2

User Output
Enter the first string:
Hello
Enter the second string:
World
Concatenated string: HelloWorld
Length of concatenated string: 10
Copied string: HelloWorld

S.No: 52	Exp. Name: ***Counting the Number of Words in a Sentence***	**Date: 2026-10-15**

ID: 2602050330
2026-2030-Batch2-I1
Manipal University Jaipur

Aim:

Write a program to count the number of words in a sentence.

Input Format:

- The program should display the prompt:

`Enter a sentence:`

- The user should enter a sentence (a string, possibly containing spaces), on the same line as the prompt.

Output Format:

- The program should display the number of words in the sentence in the format:

`Number of words in the sentence: <words>`

Source Code:

numberofWords.c

```
#include <stdio.h>

int main() {
        char s[100], count=0;
        printf("Enter a sentence: ");
        scanf(" %[^\n]", s);

        for (int i=0; s[i]; i++) {
                if (s[i] != ' ' && (i == 0 || s[i-1] == ' '))
                        count++;
        }

        printf("Number of words in the sentence: %d\n", count);
        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter a sentence:

Number of words in the sentence: 2

Test Case - 2
User Output
Enter a sentence:
C Programming Problems
Number of words in the sentence: 3

S.No: 53	Exp. Name: ***Reversing a String***	Date: 2026-10-15

ID: 2602050330

2026-2030-Batch2-I1

Manipal University Jaipur

Aim:

Write a program to reverse a string.

Input Format:

- The program should display the prompt:

`Enter a string:`

- The user should enter a string (possibly containing spaces), on the same line as the prompt.

Output Format:

- The program should display the reversed string in the format:

`Reversed string: <reversed string>`

Source Code:

stringReverse.c

```
#include <stdio.h>
#include <string.h>

int main()
{
        char str[100];
        printf("Enter a string: ");
        scanf("%[^\n]", str);

        int len = strlen(str);

        printf("Reversed string: ");
        for (int i = len - 1; i >= 0; i--)
                printf("%c", str[i]);
        printf("\n");

        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1

User Output
Enter a string:
C programming
Reversed string: gnimmargorp C

Test Case - 2
User Output
Enter a string:
racecar
Reversed string: racecar

S.No: 54	Exp. Name: ***Finding the Length of a String Without Using a Predefined Function***	**Date: 2026-10-15**

ID: 2602050330
2026-2030-Batch2-I1
Manipal University Jaipur

Aim:

Write a program to find the string length of a string without using the predefined function.

Input Format:

- The program should display the prompt:

`Enter a string:`

- The user should enter a string (possibly containing spaces), on the same line as the prompt.

Output Format:

- The program should display the length of the string in the format:

`Length of the given string: <length>`

Source Code:

stringLength.c

```
#include <stdio.h>

int main()
{
        char s[100];
        int len=0;
        printf("Enter a string: ");
        scanf("%[^\n]", s);

        while (s[len] != '\0')
                len++;

        printf("Length of the given string: %d\n", len);
        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output

Enter a string:
hello
Length of the given string: 5

Test Case - 2
User Output
Enter a string:
CodeTantra
Length of the given string: 10

S.No: 55	Exp. Name: ***Finding a Substring in a Given String***	**Date: 2026-10-15**

Aim:

Write a program to find the substring of a given string.

Input Format:

- The program should display the prompt:

```
Enter a string:
```

- The user should enter a string (possibly containing spaces), on the same line as the prompt.
- The program should then display the prompt:

```
Enter a substring:
```

- The user should enter a substring (possibly containing spaces), on the same line as the prompt.

Output Format:

- If the substring is found in the string, the program should display:

```
Substring found at index <index>
```

- If the substring is not found, the program should display:

```
Substring not found in the string
```

Source Code:

subStringCheck.c

```
#include <stdio.h>
#include <string.h>

int main()
{
        char s[100], sub[100];
        int i, j, found=-1;

        printf("Enter a string: ");
        scanf(" %[^\n]", s);

        printf("Enter a substring: ");
        scanf(" %[^\n]", sub);

        for (i=0; i<strlen(s); i++) {
                for (j=0; j<strlen(sub); j++) {
                        if(s[i+j] != sub[j])
                                break;
                }
                if (j==strlen(sub)) {
                        found=i;
                        break;
                }
        }

        if (found != -1)
                printf("Substring found at index %d\n", found);
        else
                printf("Substring not found in the string\n");

        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter a string:
hello world
Enter a substring:

world
Substring found at index 6

Test Case - 2
User Output
Enter a string:
hello
Enter a substring:
xyz
Substring not found in the string

S.No: 56	Exp. Name: ***Checking if a String is a Palindrome***	**Date: 2026-10-15**

Aim:

Write a program to check if the given string is a palindrome or not.

Input Format:

- The program should display the prompt:

```
Enter a string:
```

- The user should enter a string (possibly containing spaces), on the same line as the prompt.

Output Format:

- If the string is a palindrome, the program should display:

```
The string is a palindrome
```

- If the string is not a palindrome, the program should display:

```
The string is not a palindrome
```

Source Code:

palindromCheck.c

```
#include <stdio.h>
#include <string.h>

int main()
{
        char s[100];
        int n, flag=1;
        printf("Enter a string: ");
        scanf("%[^\n]", s);

        n = strlen(s);
        for (int i=0; i<n/2; i++) {
                if (s[i] != s[n-i-1]) {
                        flag=0;
                        break;
                }
        }

        if (flag)
                printf("The string is a palindrome\n");
        else
                printf("The string is not a palindrome\n");

        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter a string:
hello
The string is not a palindrome

Test Case - 2
User Output
Enter a string:

```
The string is a palindrome
```

S.No: 57	Exp. Name: ***Factorial Calculation Using a Function***	**Date: 2026-10-15**

Aim:

Write a program to find the factorial of a number using a function.(Ex: 5! =5*4*3*2*1. Use a function **Fact** to evaluate factorial & print the result).

Input Format:

- The program should display the prompt:

```
Enter a number:
```

- The user should enter a single integer n on the same line as the prompt.

Output Format:

- If n is negative, the program should display:

```
Factorial is not defined for negative numbers
```

- Otherwise, the program should display the factorial in the following format:

```
Factorial of <n> = factorial
```

Source Code:

factorial.c

```
#include <stdio.h>

long long fact (int n) {
        long long f=1;
        for (int i=1; i<=n; i++) {
                f=f*i;
        }
        return f;
}

int main() {
        int n;

        printf("Enter a number: ");
        scanf("%d", &n);

        if (n<0) {
                printf("Factorial is not defined for negative
numbers\n");
        } else {
                printf("Factorial of %d = %lld\n", n, fact(n));
        }

        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter a number:
0
Factorial of 0 = 1

Test Case - 2
User Output
Enter a number:

Factorial is not defined for negative numbers

Test Case - 3
User Output
Enter a number:
5
Factorial of 5 = 120

S.No: 58	Exp. Name: ***Maximum Element in an Array Using a Function***	**Date: 2026-10-15**

ID: 2602050330
2026-2030-Batch2-I1
Manipal University Jaipur

Aim:

Write a program to find the maximum of a given set of numbers using functions. (Use a function **Max** and return the result to the main function)

Input Format:

- The program should display the prompt:

```
Enter number of elements:
```

- The user should enter a single integer n, representing the number of elements in the array, on the same line as the prompt.
- The program should then display the prompt:

```
Enter <n> numbers:
```

- The user should enter n integers, separated by spaces, on the next line after the prompt.

Output Format:

- The program should display the maximum value among the entered array elements in the following format:

```
Maximum number = <max>
```

- where max is the largest element in the array.

Source Code:

Max.c

```
#include <stdio.h>

int max(int a[], int n) {
        int max = a[0];
        for (int i=1; i<n; i++) {
                if (a[i] > max)
                        max = a[i];
        }
        return max;
}

int main()
{
        int n, a[100];

        printf("Enter number of elements: ");
        scanf("%d", &n);

        printf("Enter %d numbers:\n", n);
        for (int i=0; i<n; i++)
                scanf(" %d", &a[i]);

        printf("Maximum number = %d\n", max(a, n));

        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter number of elements:
1
Enter 1 numbers:
50
Maximum number = 50

Test Case - 2

User Output
Enter number of elements:
4
Enter 4 numbers:
-10 -25 -5 -30
Maximum number = -5

Test Case - 3
User Output
Enter number of elements:
5
Enter 5 numbers:
10 25 25 15 5
Maximum number = 25

S.No: 59	Exp. Name: ***GCD Using Recursion***	Date: 2026-10-15

Aim:

Write a program to find the GCD of two numbers recursively. (Ex: GCD of 9, 24is 3)

Input Format:

- The program should display the prompt:

```
Enter two numbers:
```

- The user should enter two integers a and b, separated by a space, on the same line as the prompt.

Output Format:

- The program should display the Greatest Common Divisor (GCD) of the two numbers in the following format:

```
GCD of <a> and <b> = <gcd>
```

- where gcd is the GCD of a and b.

Source Code:

gcd.c

```
#include <stdio.h>

int gcd(int a, int b) {
        if (b==0)
                return a;
        return gcd(b, a%b);
}

int main()
{
        int a, b;
        printf("Enter two numbers: ");
        scanf("%d%d", &a, &b);
        printf("GCD of %d and %d = %d\n", a, b, gcd(a, b));
        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1

User Output
Enter two numbers:
12 12
GCD of 12 and 12 = 12

Test Case - 2
User Output
Enter two numbers:
0 15
GCD of 0 and 15 = 15

Test Case - 3
User Output
Enter two numbers:
8 15
GCD of 8 and 15 = 1

S.No: 60	Exp. Name: ***First N Prime Numbers Using a Function***	**Date: 2026-10-15**

ID: 2602050330
2026-2030-Batch2-I1
Manipal University Jaipur

Aim:

Write a function to check whether the given number is prime or not. Using this function, generate the first n prime numbers using the above function.

Input Format:

- The program should display the prompt:

```
Enter n:
```

- The user should enter a single integer n representing the number of prime numbers to be generated, on the same line as the prompt.

Output Format:

- The program should display the message:

```
First <n> prime numbers are:
```

- This should be followed on a new line by the first n prime numbers, each separated by a space.

Source Code:

Prime.c

```
#include <stdio.h>

int isPrime(int num) {
        if (num < 2)
                return 0;
        for (int i=2; i*i<=num; i++) {
                if (num%i==0)
                        return 0;
        }
        return 1;
}

int main() {
        int n, count=0, num=2;

        printf("Enter n: ");
        scanf("%d", &n);

        printf("First %d prime numbers are:\n", n);
        while (count<n) {
                if (isPrime(num)) {
                        printf("%d ", num);
                        count++;
                }
                num++;
        }
        printf("\n");

        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter n:
1
First 1 prime numbers are:
2

Test Case - 2
User Output
Enter n:
2
First 2 prime numbers are:
2 3

Test Case - 3
User Output
Enter n:
10
First 10 prime numbers are:
2 3 5 7 11 13 17 19 23 29

S.No: 61	Exp. Name: ***First N Fibonacci Terms Using Recursion***	**Date: 2026-10-15**

ID: 2602050330

2026-2030-Batch2-I1

Manipal University Jaipur

Aim:

Write a function to generate the n^{th} Fibonacci term using recursion. Print first N Fibonacci terms using this function. Hint: (Fibonacci series is 0, 1, 1, 2, 3, 5, 8,)

Input Format:

- The program should display the prompt:

```
Enter N:
```

- The user should enter a single integer n, representing the number of Fibonacci terms to be displayed, on the same line as the prompt.

Output Format:

- The program should display the message:

```
First <n> Fibonacci terms are:
```

- This should be followed on a new line by the first n Fibonacci terms, with each term separated by a space.

Source Code:

Fibonacci.c

```
#include <stdio.h>

int fib(int n) {
        if (n==0)
                return 0;
        if (n==1)
                return 1;
        return fib(n-1) + fib(n-2);
}

int main() {
        int n;

        printf("Enter N: ");
        scanf("%d", &n);

        printf("First %d Fibonacci terms are:\n", n);
        for (int i=0; i<n; i++)
                printf("%d ", fib(i));
        printf("\n");

        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter N:
1
First 1 Fibonacci terms are:
0

Test Case - 2
User Output
Enter N:
2

0 1

Test Case - 3
User Output
Enter N:
10
First 10 Fibonacci terms are:
0 1 1 2 3 5 8 13 21 34

S.No: 62	Exp. Name: ***Checking Whether a String is a Palindrome***	**Date: 2026-10-15**

Aim:

Write a program to check if the given string is a palindrome or not, using the string handling function.

Input Format:

• The program should display the prompt:

```
Enter a string:
```

• The user should enter a string on the same line as the prompt.

Output Format:

• If the given string is a palindrome, the program should display:

```
The given string is a palindrome
```

• Otherwise, the program should display:

```
The given string is not a palindrome
```

Source Code:

Palindrome.c

```
#include <stdio.h>
#include <string.h>

int isPal(char s[]) {
        int n = strlen(s);
        for (int i=0; i<n/2; i++) {
                if (s[i] != s[n-1-i])
                        return 0;
        }
        return 1;
}

int main() {
        char s[100];

        printf("Enter a string: ");
        scanf("%[^\n]", s);

        if (isPal(s))
                printf("The given string is a palindrome\n");
        else
                printf("The given string is not a palindrome\n");

        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter a string:
a
The given string is a palindrome

Test Case - 2
User Output
Enter a string:

The given string is a palindrome

Test Case - 3
User Output
Enter a string:
hello
The given string is not a palindrome

S.No: 63	Exp. Name: ***Sorting Names in Alphabetical Order Using Functions***	**Date: 2026-10-15**

ID: 2602050330
2026-2030-Batch2-I1
Manipal University Jaipur

Aim:

Write a function **Sort** for sorting a list of names which will use a function compare to compare two names. (Selection /bubble Sort may be used).

Input Format:

- The program should display the prompt:

```
Enter number of names:
```

- The user should enter a single integer n, representing the number of names, on the same line as the prompt.
- The program should then display the prompt:

```
Enter <n> names:
```

- The user should enter n names, with each name entered on a separate line after the prompt.

Output Format:

- The program should display the message:

```
Names in alphabetical order:
```

- This should be followed on a new line by the n names arranged in alphabetical order, with each name printed on a separate line.

Source Code:

sorting.c

```
#include <stdio.h>
#include <string.h>

int compare(char a[], char b[]) {
        return strcmp(a, b);
}

void Sort(char names[][100], int n) {
        char temp[100];
        for (int i=0; i<n; i++) {
                for (int j=i+1; j<n; j++) {
                        if (compare(names[i], names[j]) > 0) {
                                strcpy(temp, names[i]);
                                strcpy(names[i], names[j]);
                                strcpy(names[j], temp);
                        }
                }
        }
}

int main()
{
        int n;
        char names[100][100];

        printf("Enter number of names: ");
        scanf("%d", &n);

        printf("Enter %d names:\n", n);
        for (int i=0; i<n; i++) {
                scanf(" %[^\n]", names[i]);
        }

        Sort(names, n);

        printf("Names in alphabetical order:\n");
        for (int i=0; i<n; i++) {
                printf("%s\n", names[i]);
        }

        return 0;
}
```

Manipal University Jaipur 2026-2030-Batch2-I1 ID: 2602050330 Page No: 174

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter number of names:
4
Enter 4 names:
Anita
Bobby
David
Zara
Names in alphabetical order:
Anita
Bobby
David
Zara

Test Case - 2
User Output
Enter number of names:
4
Enter 4 names:
John
John
Alice
Bob
Names in alphabetical order:
Alice
Bob
John
John

Test Case - 3
User Output
Enter number of names:
6

Arun
Zoya
Bharat
Neha
Chetan
Names in alphabetical order:
Arun
Bharat
Chetan
Meena
Neha
Zoya

S.No: 64	Exp. Name: ***Calculating the Sum of Two Integers***	**Date: 2026-10-15**

Aim:

Write a program to access two integers using pointers and add them.

Input Format:

- The program should display the prompt:

```
Enter two integers:
```

- The user should enter two integers a and b, separated by a space, on the same line as the prompt.

Output Format:

- The program should display the sum of the two integers in the following format:

```
Sum = <sum>
```

where <sum> is the result obtained by adding the two input integers using pointers.

Note:

- Have a newline character (\n) at the end of every output line.

Source Code:

Sum.c

```
#include <stdio.h>

int main()
{
        int a, b;
        int *p1 = &a, *p2 = &b;

        printf("Enter two integers: ");
        scanf("%d %d", p1, p2);

        printf("Sum = %d\n", *p1 + *p2);
        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output

Enter two integers:
10 20
Sum = 30

Test Case - 2
User Output
Enter two integers:
0 0
Sum = 0

Test Case - 3
User Output
Enter two integers:
15 -5
Sum = 10

S.No: 65	Exp. Name: ***Finding the Greatest and Smallest of Three Numbers***	**Date: 2026-10-15**

Aim:

Write a program to find out the greatest and the smallest among the three numbers using pointers.

Input Format:

- The program should display the prompt:

```
Enter three numbers:
```

- The user should enter three integers a, b, and c, separated by spaces, on the same line as the prompt.

Output Format:

- The program should display the greatest number in the following format:

```
Greatest = <greatest>
```

- The program should then display the smallest number in the following format:

```
Smallest = <smallest>
```

where <greatest> and <smallest> are the largest and smallest values among the three input integers, respectively.

Note:

- Have a newline character (\n) at the end of every output line.

Source Code:

Greatest_Smallest.c

```c
#include <stdio.h>

int main() {
        int a, b, c;
        int *p1 = &a, *p2 = &b, *p3=&c;
        int *greatest, *smallest;

        printf("Enter three numbers: ");
        scanf("%d %d %d", p1, p2, p3);

        greatest=p1;
        smallest=p1;

        if (*p2 > *greatest)
                greatest = p2;
        if (*p3 > *greatest)
                greatest = p3;

        if (*p2 < *smallest)
                smallest = p2;
        if (*p3 < *smallest)
                smallest = p3;

        printf("Greatest = %d\n", *greatest);
        printf("Smallest = %d\n", *smallest);

        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter three numbers:
10 25 15
Greatest = 25
Smallest = 10

User Output
Enter three numbers:
8 8 8
Greatest = 8
Smallest = 8

Test Case - 3
User Output
Enter three numbers:
-10 -5 -20
Greatest = -5
Smallest = -20

S.No: 66	Exp. Name: ***Finding the Length of a String***	Date: 2026-10-15

ID: 2602050330
2026-2030-Batch2-I1
Manipal University Jaipur

Aim:

Write a program to determine the length of a character string using a pointer.

Input Format:

- The program should display the prompt:

`Enter a string:`

- The user should enter a string on the same line as the prompt.

Output Format:

- The program should display the length of the entered string in the following format:

`Length of string = <length>`

where <length> represents the number of characters in the string, excluding the newline character.

Note:

- Have a newline character (\n) at the end of every output line.

Source Code:

length.c

```
#include <stdio.h>

int main() {
        char s[100], *p=s;
        int len=0;

        printf("Enter a string: ");
        scanf("%[^\n]", s);

        while (*p != '\0') {
                len++;
                p++;
        }

        printf("Length of string = %d\n", len);

        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter a string:
A
Length of string = 1

Test Case - 2
User Output
Enter a string:
Hello
Length of string = 5

Test Case - 3
User Output
Enter a string:
C Programming
Length of string = 13

S.No: 67	Exp. Name: ***Calculating the Sum of Array Elements***	**Date: 2026-10-15**

ID: 2602050330
2026-2030-Batch2-I1
Manipal University Jaipur

Aim:

Write a program to compute the sum of all elements stored in an array using a pointer.

Input Format:

- The program should display the prompt:

```
Enter number of elements:
```

- The user should enter a single integer n, representing the number of elements in the array, on the same line as the prompt.
- The program should then display the prompt:

```
Enter n elements:
```

- The user should enter n integers, separated by spaces, on the next line after the prompt.

Output Format:

- The program should display the sum of all the array elements in the following format:

```
Sum of array elements = <sum>
```

where <sum> is the sum of all n array elements calculated using a pointer.

Note:

- Have a newline character (\n) at the end of every output line.

Source Code:

SumOfArrays.c

```
#include <stdio.h>

int main()
{
        int n, a[100], sum=0, *p=a;
        printf("Enter number of elements: ");
        scanf("%d", &n);

        printf("Enter %d elements:\n", n);
        for (int i=0; i<n; i++)
                scanf("%d", p+i);

        for (int i=0; i<n; i++)
                sum+=*(p+i);

        printf("Sum of array elements = %d\n", sum);
        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter number of elements:
1
Enter 1 elements:
25
Sum of array elements = 25

Test Case - 2
User Output
Enter number of elements:
5
Enter 5 elements:
10 20 30 40 50
Sum of array elements = 150

Manipal University Jaipur 2026-2030-Batch2-I1 ID: 2602050330 Page No: 185

Test Case - 3
User Output
Enter number of elements:
4
Enter 4 elements:
10 -5 20 -10
Sum of array elements = 15

S.No: 68	Exp. Name: ***Finding the First Occurrence of a Substring in a String***	**Date: 2026-10-15**

Aim:

Write a program to determine whether a substring (string 1) is in the main string or not. If present, return the pointer of the first occurrence

Input Format:

- The program should display the prompt:

```
Enter main string:
```

- The user should enter a main string on the same line as the prompt.
- The program should then display the prompt:

```
Enter substring:
```

- The user should enter the substring to be searched on the same line as the prompt.

Output Format:

- If the substring is found in the main string, the program should display:

```
Substring found
First occurrence: <remaining_string>
```

where <remaining_string> is the portion of the main string starting from the first occurrence of the substring.

- If the substring is not found, the program should display:

```
Substring not found
```

Note:

- Have a newline character (\n) at the end of every output line.

Source Code:

First_Occurrence.c

```
#include <stdio.h>
#include <string.h>

int main()
{
        char s[100], sub[100], *p;

        printf("Enter main string: ");
        scanf("%[^\n]", s);

        printf("Enter substring: ");
        scanf(" %[^\n]", sub);

        p=strstr(s, sub);

        if (p != NULL) {
                printf("Substring found\n");
                printf("First occurrence: %s\n", p);
        } else {
                printf("Substring not found\n");
        }

        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter main string:
Hello World
Enter substring:
Hello
Substring found
First occurrence: Hello World

Test Case - 2

Enter main string:
I love programming
Enter substring:
love
Substring found
First occurrence: love programming

Test Case - 3
User Output
Enter main string:
Hello World
Enter substring:
Java
Substring not found

S.No: 69	Exp. Name: ***Displaying Personal Information***	**Date: 2026-10-20**

ID: 2602050330
2026-2030-Batch2-I1
Manipal University Jaipur

Aim:

Write a program to define a structure **personal** that would contain the person's name, date of joining, and salary. Using this structure write a program to read this information for one person from the keyboard and print the same on the screen.

Input Format:

- The program should display the prompt:

```
Enter name:
```

- The user should enter the person's name on the same line as the prompt.
- The program should then display the prompt:

```
Enter date of joining (DD/MM/YYYY):
```

- The user should enter the date of joining in the format DD/MM/YYYY on the same line as the prompt.
- The program should then display the prompt:

```
Enter salary:
```

- The user should enter the salary as a floating-point value on the same line as the prompt.

Output Format:

- The program should display the heading:

```
Personal Information
```

- This should be followed on separate lines by the person's details in the following format:

```
Name: name
Date of Joining: date
Salary: salary
```

- The salary should be displayed with two decimal places.

Note:

- Have a newline character (\n) at the end of every output line.

Source Code:

PersonalInfo.c

```c
#include <stdio.h>

struct personal {
        char name[500];
        char doj[50];
        float salary;
};

int main()
{
        struct personal p;
        printf("Enter name: ");
        scanf("%[^\n]", p.name);
        printf("Enter date of joining (DD/MM/YYYY): ");
        scanf("%s", p.doj);
        printf("Enter salary: ");
        scanf("%f", &p.salary);
        printf("Personal Information\n");
        printf("Name: %s\n", p.name);
        printf("Date of Joining: %s\n", p.doj);
        printf("Salary: %.2f\n", p.salary);
        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter name:
John Smith
Enter date of joining (DD/MM/YYYY):
15/06/2024
Enter salary:
45000
Personal Information
Name: John Smith
Date of Joining: 15/06/2024
Salary: 45000.00

Test Case - 2
User Output
Enter name:
Priya Sharma
Enter date of joining (DD/MM/YYYY):
01/01/2023
Enter salary:
32500.50
Personal Information
Name: Priya Sharma
Date of Joining: 01/01/2023
Salary: 32500.50

S.No: 70	Exp. Name: ***Sorting Students by Total Marks***	**Date: 2026-10-20**

Aim:

Write a program to create an array of student structures to store the roll no, name, and marks in 3 subjects. Input the details of n students into the array and display roll no., name, and total marks of each student in decreasing order of total marks.

Input Format:

- The program should display the prompt:

```
Enter number of students:
```

- The user should enter a single integer n, representing the number of students, on the same line as the prompt.
- For each student, the program should display:

```
Enter details of Student i
Roll No:
```

where i represents the student number.

- The user should enter the student's roll number on the same line as the prompt.
- The program should then display:

```
Name:
```

- The user should enter the student's name on the same line as the prompt.
- The program should then display:

```
Enter marks in 3 subjects:
```

- The user should enter three floating-point marks, separated by spaces, on the same line as the prompt.
- The above process should be repeated for all n students.

Output Format:

- The program should display a separator followed by the heading:

```
Roll No Name        Total Marks
```

- This should be followed on separate lines by the details of each student, arranged in **decreasing order of total marks**, in the format:

```
rollNo      name            total
```

- The total marks should be displayed with **two decimal places**.

Note:

- In the heading, use one tab (\t) between "Roll No" and "Name", and two tabs (\t\t) between "Name" and "Total Marks".
- For each student record, use two tabs (\t\t) between <roll_no> and <name>.
- The name must be left-aligned within a field width of 15 characters, followed by one space and <total_marks>.
- Have a newline character (\n) at the end of every output line.

Source Code:

Student_Marks.c

```
#include <stdio.h>

struct student {
        int roll;
        char name[100];
        float total;
};

int main() {
        int n;
        printf("Enter number of students: ");
        if (scanf("%d", &n) != 1) {
                return 0;
        }
        struct student s[n], t;
        for (int i=0; i<n; i++) {
                float m1, m2, m3;
                printf("Enter details of Student %d\nRoll No: ", i+1);
                scanf("%d", &s[i].roll);
                printf("Name: ");
                scanf("%s", s[i].name);
                printf("Enter marks in 3 subjects: ");
                scanf("%f %f %f", &m1, &m2, &m3);
                s[i].total = m1+m2+m3;
        }

        for (int i=0; i<n; i++) {
                for (int j=i+1; j<n; j++) {
                        if (s[i].total < s[j].total) {
                                t = s[i];
                                s[i] = s[j];
                                s[j] = t;
                        }
                }
        }

        printf("Roll No\tName\t\tTotal Marks\n");
        for (int i=0; i<n; i++) {
                printf("%d\t\t%-15s %.2f\n", s[i].roll, s[i].name,
s[i].total);
        }
        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter number of students:
2
Enter details of Student 1
Roll No:
301
Name:
Sam
Enter marks in 3 subjects:
85.5 76.5 90
Enter details of Student 2
Roll No:
302
Name:
Neha
Enter marks in 3 subjects:
92.5 88 91.5
Roll No Name Total Marks
302 Neha 272.00
301 Sam 252.00

Test Case - 2
User Output
Enter number of students:
3
Enter details of Student 1
Roll No:
101
Name:
John
Enter marks in 3 subjects:
60 70 80
Enter details of Student 2
Roll No:
102

```
Mary
Enter marks in 3 subjects:
90 85 95
Enter details of Student 3
Roll No:
103
Name:
Bob
Enter marks in 3 subjects:
75 80 90
Roll No Name            Total Marks
102             Mary            270.00
103             Bob             245.00
101             John            210.00
```

S.No: 71	Exp. Name: ***Finding Employees with Above - Average Net Salary***	**Date: 2026-10-20**

Aim:

Write a program to create an array of employee structures to store emp-no, name, basic salary, and HRA. Input the details of n employees and display emp-no, name, basic, HRA, and net salary. Display the details of all employees whose net salary is more than the average net salary of all employees.

Input Format:

- The program should display the prompt:

```
Enter number of employees:
```

- The user should enter a single integer n, representing the number of employees, on the same line as the prompt.
- For each employee, the program should display:

```
Enter details of Employee i
Employee No.:
```

where i represents the employee number.

- The user should enter the employee number on the same line as the prompt.
- The program should then display:

```
Name:
```

- The user should enter the employee's name on the same line as the prompt.
- The program should then display:

```
Basic Salary:
```

- The user should enter the employee's basic salary as a floating-point value on the same line as the prompt.
- The program should then display:

```
HRA:
```

- The user should enter the HRA as a floating-point value on the same line as the prompt.
- The above process should be repeated for all n employees.

Output Format:

- The program should display the average net salary in the following format:

```
Average Net Salary = <average>
```

where <average> is the average of the net salaries of all employees, displayed with two decimal places.

- The program should then display:

```
Employees having net salary more than average:
Emp No  Name        Basic       HRA     Net Salary
```

- This should be followed on separate lines by the details of employees whose net salary is greater than the average net salary, in the following format:

```
emp_no    name    basic    hra    net_salary
```

- The Basic, HRA, and Net Salary values should be displayed with two decimal places.

Note:

- Have a newline character (\n) at the end of every output line.

Source Code:

EmployeesSalary.c

```
#include <stdio.h>

struct Employee {
        int id;
        char name[50];
        float basic, hra, net;
};
int main() {
        int n;
        float total=0;
        printf("Enter number of employees: ");
        scanf("%d", &n);
        struct Employee emp[n];
        for (int i=0; i<n; i++) {
                printf("Enter details of Employee %d\n", i+1);
                printf("Employee No.: ");
                scanf("%d", &emp[i].id);
                printf("Name: ");
                scanf(" %[^\n]", emp[i].name);
                printf("Basic Salary: ");
                scanf("%f", &emp[i].basic);
                printf("HRA: ");
                scanf("%f", &emp[i].hra);
                emp[i].net = emp[i].basic+emp[i].hra;
                total += emp[i].net;
        }

        float avg = total/n;
        printf("Average Net Salary = %.2f\n", avg);
        printf("Employees having net salary more than average:\n");
        printf("Emp No\tName\t\tBasic\t\tHRA\t\tNet Salary\n");
        for (int i=0; i<n; i++) {
                if (emp[i].net>avg) {
                        printf("%d\t%-15s %.2f\t%.2f\t\t%.2f\n",
emp[i].id, emp[i].name, emp[i].basic, emp[i].hra, emp[i].net);
                }
        }

        return 0;
}
```


ID: 2602050330
2026-2030-Batch2-I1
Manipal University Jaipur

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter number of employees:
2
Enter details of Employee 1
Employee No.:
101
Name:
Alice
Basic Salary:
30000
HRA:
5000
Enter details of Employee 2
Employee No.:
102
Name:
Bob
Basic Salary:
40000
HRA:
10000
Average Net Salary = 42500.00
Employees having net salary more than average:
Emp No Name Basic HRA Net Salary
102 Bob 40000.00 10000.00 50000.00

Test Case - 2
User Output
Enter number of employees:
4
Enter details of Employee 1
Employee No.:
501
Name:

28000.50
HRA:
4500.25
Enter details of Employee 2
Employee No.:
502
Name:
Priya
Basic Salary:
42000.75
HRA:
7500.50
Enter details of Employee 3
Employee No.:
503
Name:
Rahul
Basic Salary:
35000.25
HRA:
5000.00
Enter details of Employee 4
Employee No.:
501
Name:
Sneha
Basic Salary:
50000.00
HRA:
8500.00
Average Net Salary = 45125.56
Employees having net salary more than average:

Emp No	Name	Basic	HRA	Net Salary
502	Priya	42000.75	7500.50	49501.25
501	Sneha	50000.00	8500.00	58500.00

Test Case - 3

User Output

```
Enter details of Employee 1
Employee No.:
101
Name:
John
Basic Salary:
60000
HRA:
5000
Average Net Salary = 65000.00
Employees having net salary more than average:
Emp No  Name            Basic           HRA             Net Salary
```

S.No: 72	Exp. Name: ***Calculating the Date After Adding 45 Days***	**Date: 2026-10-20**

Aim:

Write a program to create a structure named **Date** having day, month, and year as its elements. Store the current date in the structure. Now add 45 days to the current date and display the final date.

Input Format:

- The program should display the prompt:

```
Enter current date (DD MM YYYY):
```

- The user should enter three integers representing the day, month, and year of the current date, separated by spaces, on the same line as the prompt.
- The date should be entered in the order DD MM YYYY.

Output Format:

- The program should display the date obtained after adding 45 days to the given date in the following format:

```
Date after adding 45 days: DD/MM/YYYY
```

- The day and month should be displayed using two digits (for example, 05/03/2024).

Note:

- Have a newline character (\n) at the end of every output line.

Source Code:

DateCalculation.c

```
#include <stdio.h>

struct Date {
        int day, month, year;
};

int main() {
        struct Date d;
        printf("Enter current date (DD MM YYYY): ");
        scanf("%d%d%d", &d.day, &d.month, &d.year);
        d.day+=45;

        while (1) {
                int m=d.month, y=d.year;
                int days = (m==2) ? ((y%4==0 && y%100!=0) || y%400==0?
29:28) : ((m==4 ||m==6||m==9||m==11)?30:31);
                if (d.day<=days) {
                        break;
                }
                d.day-=days;
                if (++d.month>12) {
                        d.month = 1;
                        d.year++;
                }
        }

        printf("Date after adding 45 days: %02d/%02d/%04d\n", d.day,
d.month, d.year);
        return 0;
}
```

Execution Results - All test cases have succeeded!

Test Case - 1
User Output
Enter current date (DD MM YYYY):
20 02 2024
Date after adding 45 days: 05/04/2024

Test Case - 2
User Output
Enter current date (DD MM YYYY):
20 12 2023
Date after adding 45 days: 03/02/2024

Test Case - 3
User Output
Enter current date (DD MM YYYY):
15 01 2023
Date after adding 45 days: 01/03/2023

Test Case - 4
User Output
Enter current date (DD MM YYYY):
15 01 2024
Date after adding 45 days: 29/02/2024